一、计算机网络基础知识
1、什么是计算机网络
2、计算机网络的功能
3、计算机网络的组成
1、通信子网
2、资源子网
3、计算机网络软件
协议软件：
网络通信软件：
网络操作系统：
网络应用软件：
4、计算机网络的分类
5、计算机网络的发展过程
6、OSI七层网络模型
7、TCP/IP模型四层
8、MAC地址、IP地址、域名、DNS服务器
MAC地址：
IP地址：
域名：
DNS服务器：
二、IP地址
1、IPv4地址
2、IPv4地址的格式
3、IPv4地址的分类
4、特殊的IPv4地址
5、公网IPv4地址与私网IPv4地址
6、子网掩码
7、网关地址
8、IPv6地址
9、IPv4与IPv6的区别
地址类型：
数据包大小：
header区域字段数：
可选字段：
配置：
安全性：
与移动设备的兼容性：
主要功能：
10、IPv4或IPv6使用哪个
三、TCP协议与UDP协议
1、TCP/IP、TCP、UDP是什么
2、TCP协议介绍
TCP的主要特点：
TCP的传输机制：
TCP的首部格式：
3、TCP的连接过程
TCP三次握手的过程：
TCP四次挥手的过程：
注意：
4、UDP协议介绍
UDP的主要特点：
UDP的首部格式：
Length：
Checksum：
UDP的适用范围：

5、选择TCP还是UDP
四、套接字
1、什么是套接字
2、创建套接字
3、套接字的连接
五、IP地址转换与字节序转换
六、基于TCP协议的网络Socket通信
1、开启监听
2、等待连接
3、TCP的数据发送和接收
3、详细编程步骤
七、基于UDP协议的网络Socket通信
1、UDP协议的特点
2、UDP的数据发送和接收
3、详细编程步骤
4、UDP的连接操作
八、本地Socket通信
1、本地Socket通信与网络Socket通信的区别
2、创建Socket文件
3、基于TCP协议的本地Socket通信
4、基于UDP协议的网络Socket通信
九、Windows系统下的Socket通信
1、Windows系统的Socket介绍
2、Windows系统的Socket的库和头文件
3、Winsock库的初始化和注销
4、Windows系统的Socket相关函数
5、Windwos系统下的TCP客户端
6、Windwos系统下的UDP客户端
7、Windows系统的Socket与Linux系统的Socket的区别
十、多路复用
1、IO模型介绍
2、使用select函数实现多路复用IO
select设计不合理的地方：
select的优点是：
4、使用pselect函数实现多路复用IO
5、使用poll函数实现多路复用IO
6、使用epoll函数实现多路复IO

一、计算机网络基础知识

1、什么是计算机网络

把分布在不同地理位置的计算机与专门的网络设备用通信线路互相连成一个规模大、功能强的系统，从而使众多计算机可以方便地互相传递信息、共享软件、硬件、数据信息等。

计算机网络就是由通信线路互相连接的许多自主工作的计算机构成的的集合，它是计算机技术和通信技术相结合的产物。

2、计算机网络的功能

数据通信、资源共享、提高系统的可靠性、分布式网络处理和负载均衡。

3、计算机网络的组成

1、通信子网

- 网卡：网络接口卡或网络适配器、它负责将数据发送网络中去，也负责从网络中获取数据。
- 线缆：指的是双绞线或光纤，用于传递电信号或光信号。
- 中继器：信号在传输过程中会逐渐衰减，距离越远衰减就越严重，中继器可以对衰减的信号进行放大。
- 交换机：是一个扩大网络的器材，能为子网络中提供更多的连接端口，以便连接更多的计算机。
- 路由器：路由器是连接两个或多个网络的硬件设备，在网络间起网关的作用，是读取每一个数据包中的地址然后决定如何传送的专用智能性的网络设备。

> 它能够理解不同的协议，例如某个局域网使用的以太网协议，因特网使用的TCP/IP协议。这样，路由器可以分析各种不同类型网络传来的数据包的目的地址，把非TCP/IP网络的地址转换成TCP/IP地址，或者反之；再根据选定的路由算法把各数据包按最佳路线传送到指定位置。

2、资源子网

由连网的服务器、工作站、共享的打印机和其它设备及相关软件所组成。

3、计算机网络软件

协议软件：

它规定了计算机之间通信的准则，按网络所采用的协议层次模型(如ISO建议的开放系统互连基本参考模型）组织而成。除物理层外，其余各层协议大都由软件实现。

每层协议软件通常由一个或多个进程组成，其主要任务是完成相应层协议所规定的功能，以及与上、下层的接口功能。

网络通信软件：

主要工作是监督和控制通信工作的软件，是计算机网络软件的基础组成部分。还可以让计算机与其它计算机通信。

通信软件一般由线路缓冲区管理程序、线路控制程序以及报文管理程序组成。报文管理程序一般由接收、发送、收发记录、差错控制、开始和终了5个部分组成。

网络操作系统：

网络操作系统是用于管理网络软、硬资源，提供简单网络管理的系统软件。常见的网络操作系统有UNIX、Netware、Windows NT、Linux等。

网络应用软件：

主要任务是实现网络总体规划所规定的各项功能，提供网络服务和资源共享。网络应用软件有通用和专用之分。

通用网络应用系统适用于较广泛的领域和行业，如数据收集系统、数据转发系统和数据库查询系统等，专用网络应用系统只适用于特定的行业和领域，如银行核算、铁路控制、军事指挥等。

4、计算机网络的分类

- 1、按网络的作用范围、规模划分
  局域网、城域网、广域网
- 2、按网络传播技术划分
  广播式网络、点到点网络
- 3、传输介质划分
  有线网、无线网、微波通信、卫星通信

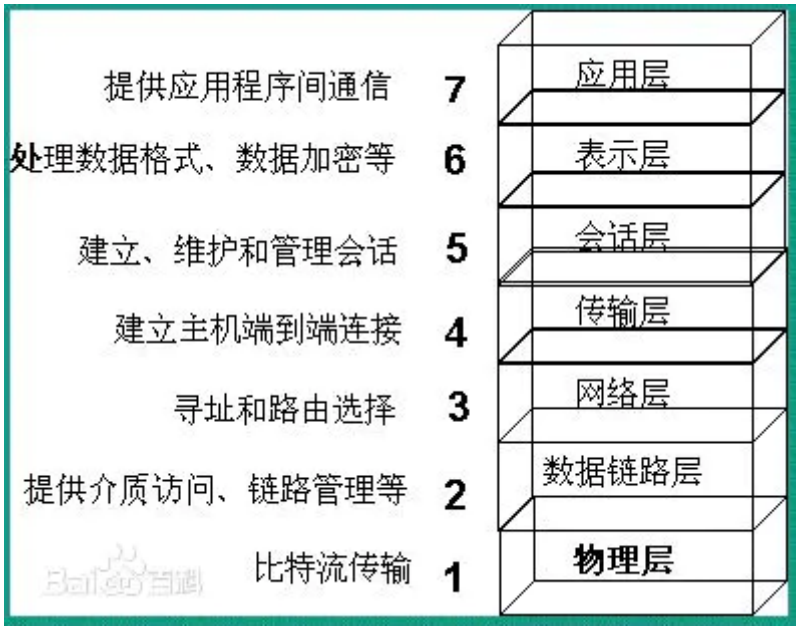
5、计算机网络的发展过程

- 1、以计算机为中心的联机系统
- 2、分组交换网络的诞生
- 3、网络体系结构与协议标准化
  20世纪80年代 ISO组织提出 开放式系统互联参考模型OSI，由于这个模型照顾到了各方和利益所以太过庞大，因此至今没有推出成熟的产品，TCP/IP是一套符合OSI标准的协议簇。
- 4、高速计算机网络4G、5G、6G

6、OSI七层网络模型

网络协议是为网络数据交换而制定的规则、约定、标准， 一个功能完备的计算机网络需要制定一整套复杂的协议集，网络协议是按层次结构来组织的，网络层次结构模型与各层协议的集合称为网络体系结构。

开放式系统互联通信参考模型（英语：Open System Interconnection Reference Model，缩写为OSI），简称为OSI模型（OSI model），一种概念模型，由国际标准化组织提出，一个试图使各种计算机在世界范围内互连为网络的标准框架。OSI将计算机网络体系结构(architecture）划分为以下七层:



物理层: 将数据转换为可通过物理介质传送的电子信号相当于邮局中的搬运工人。

数据链路层: 决定访问网络介质的方式，在此层将数据分帧，并处理流控制。本层指定拓扑结构并提供硬件寻址，相当于邮局中的装拆箱工人。

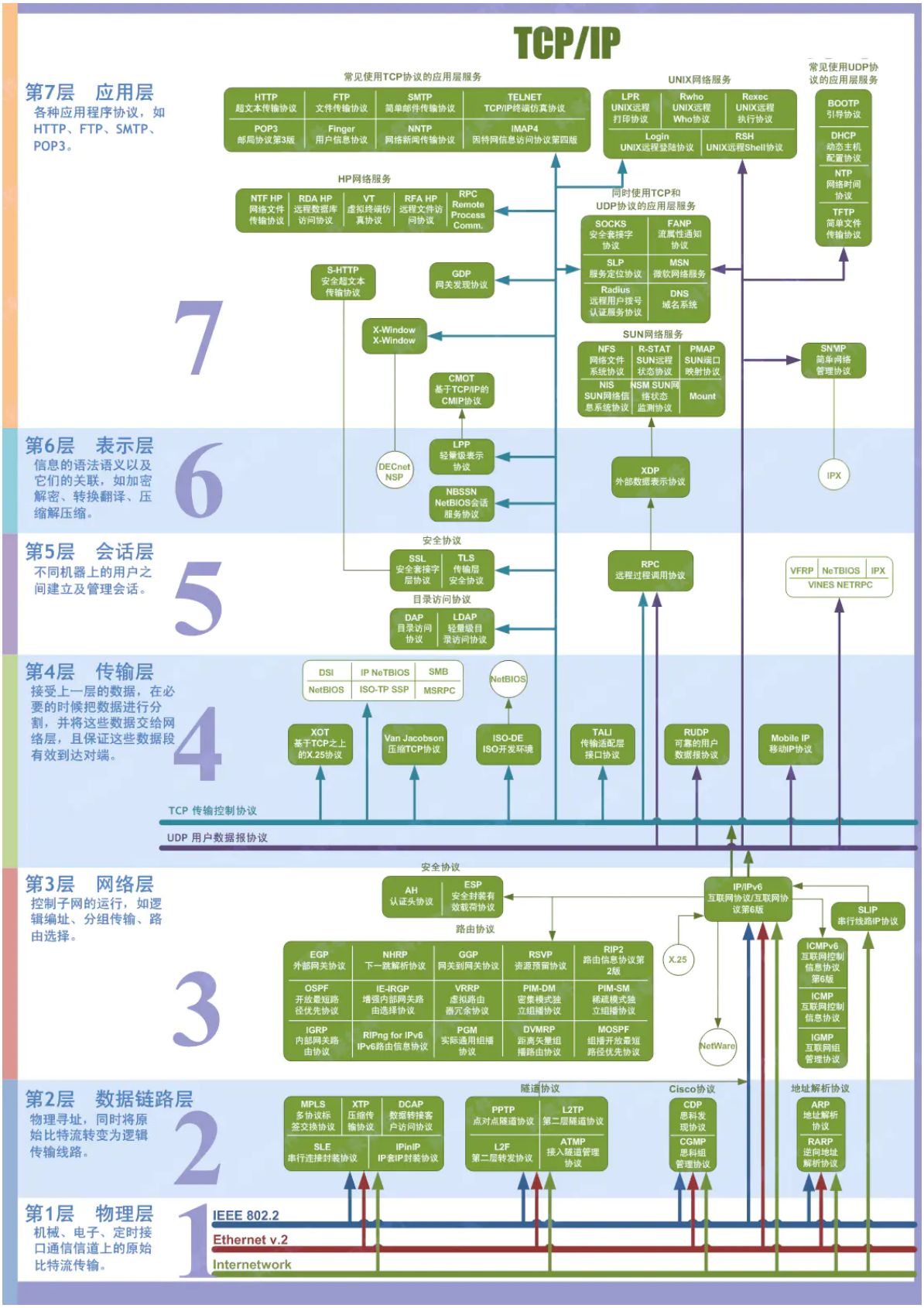
网络层: 使用权数据路由经过大型网络 相当于邮局中的排序工人。

传输层: 提供终端到终端的可靠连接 相当于公司中跑邮局的送信职员。

会话层: 允许用户使用简单易记的名称建立连接 相当于公司中收寄信、写信封与拆信封的秘书。

表示层: 协商数据交换格式 相当公司中简报老板、替老板写信的助理。

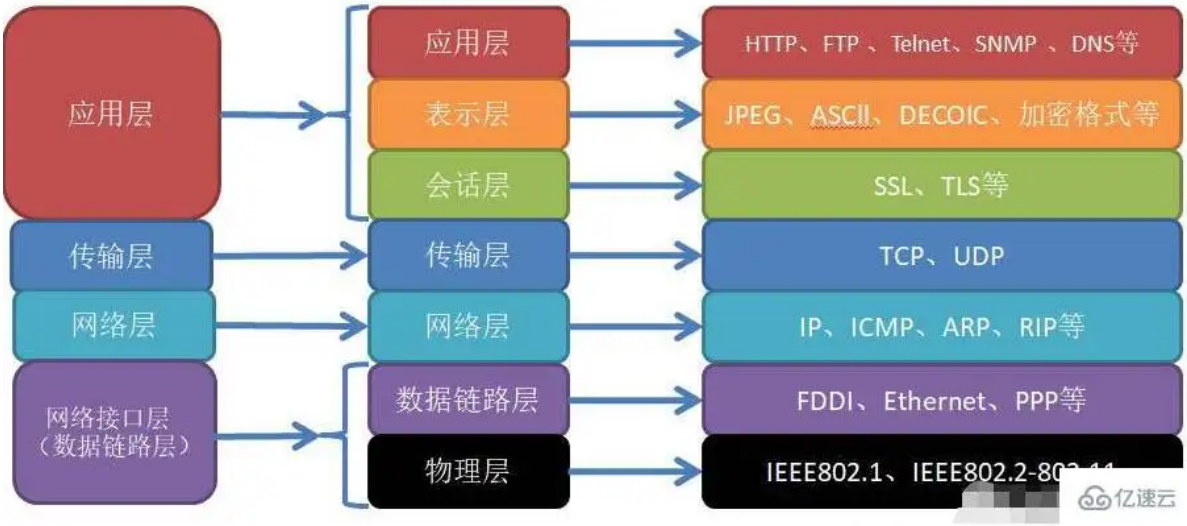
应用层: 用户的应用程序和网络之间的接口。



7、TCP/IP模型四层

既然有了OSI 模型标准，为什么还会有TCP/IP模型呢？在OSI模型开发出来之后，人们发现很多层次制定的过于详细了，比如应表会这上三层，数据在这3层是不发生任何变化的，它们仅仅是给用户提供接口以及提供一个加密功能，而且有一些层次的功能是冲突的，或者有一些层次其实没有必要区分的那么清楚，所以后来人们在使用过程中就把OSI这个模型做了一个整合，把七层整合成了四层，这就形成了TCP/IP模型。

如下图所示。它把上三层合并成了应用层，传输层、网络层保持不变，然后把物理层和数据链路层做了一个合并，合并为网络接口层。从而就把7层模型变成了四层，当然这4层模型的功能和七层模型是完全一样的。



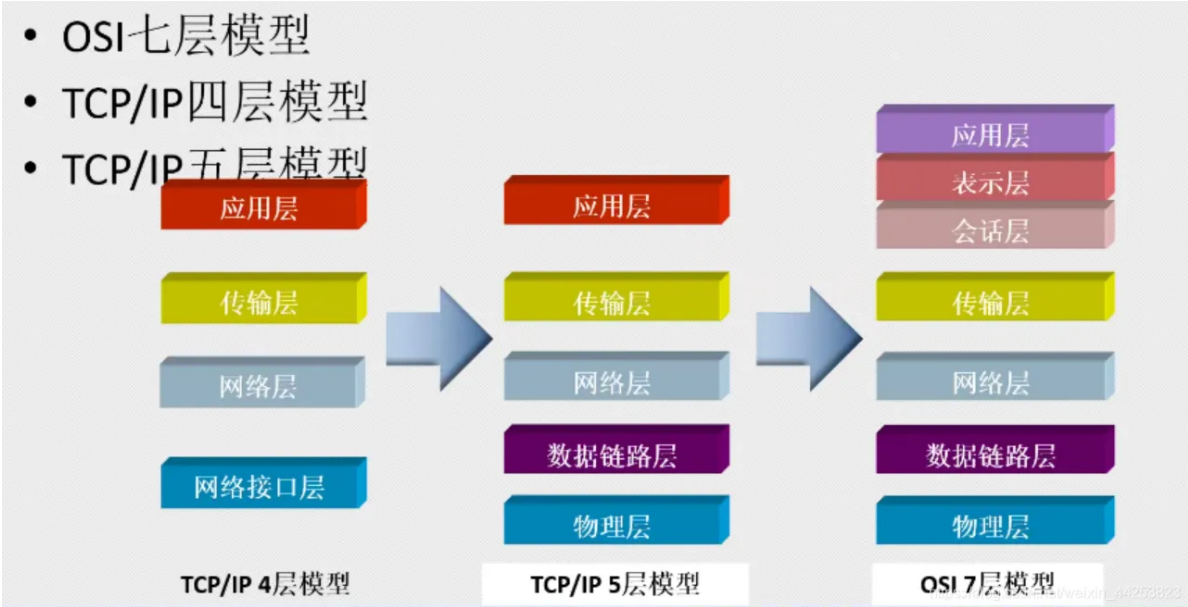
物理层：负责通信网络收发数据包

网络层：选择、流量控制、与网络拥塞问题，IP协议是该层核心。

传输层：机器之间建立用于会话的端到端连接（用于数据的传输），该层的核心协议是TCP、UDP协议。

应用层：主要为用户提供针对性的服务，该层代表性的协议有：HTTP、SMTP、FTP、TELNET。

但是有的人提出了异议，认为合并上三层可以，但是不能合并物理层和数据链路层，因为这两层的功能是完全不一样的，所以在吸取了大家的意见之后，TCP/IP模型也被修改成了五层结构，仅仅是把上三层进行了合并，下四层保持不变，如下图所示：



8、MAC地址、IP地址、域名、DNS服务器

MAC地址：

也叫物理地址，每张网卡在生产时厂家就是固化一个48位的地址（6个字节，通常表示为12个16进制数，如：00-16-EA-AE-3C-40），此地址全世界唯一，交换机路由器以此来确认网络设备位置的位址，缺点是不方便记忆、不够灵活，但它是网警打击网络犯罪的最要证据之一。

IP地址：

IP 地址是基于逻辑的，比较灵活，不受硬件的限制，也容易记忆，方便划分子网，因此在计算机网络中表面上使用IP进行通信，目前由4个不超过255的整数组成，一般用点分十进制表示（192.168.2.180）。
而在交换机、路由器中有张ARP表，一列记录MAC，另一列记录IP地址，通过ARP协议的RARP协议可以对IP地址和MAC地址进行转换。

域名：

由于普通人很难记忆大量IP地址，便有了代替IP地址的文字，也就是域名，普通人通过在浏览器中输入域名来访问各种网站，但域名最终还是要翻译成IP地址才能进行网络间通信的，域名需要从域名服务商处购买，也可以从组织或个人处转让。

DNS服务器：

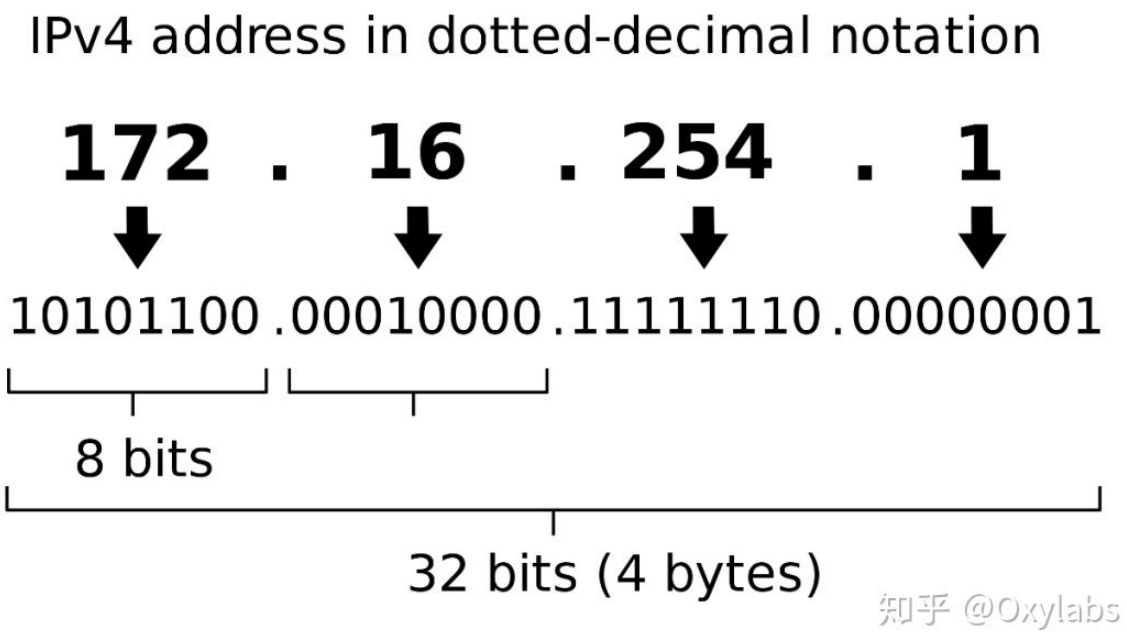
网络中有一种专门提供翻译域名服务的计算机叫DNS服务器，它负责把域名翻译成IP地址，需要在政府部分备案并缴纳一些费用才能加入DNS服务器。

全球共13台根服务器分布情况

二、IP地址

1、IPv4地址

IPv4地址的概念是在1980年代初期提出的。即使有新版本的IP地址，IPv4地址仍然是Internet用户使用最广泛的地址。由32个二进制位组成，在日常生活中IPv4地址以点分十进制表示，但在程序它就是一个网络字节序32位无符号整数。



2、IPv4地址的格式

第一部分是网络位，表示IP地址所属的网段
第二部分是主机位，用来唯一标识本网段上的某台网络设备

网络位	主机位
192.168.1	.1
11000000.10101000.00000001	.00000001

知乎 @吴某某

主机位为全0，表示一个网段，不能分配给网络设备 ，主机位为全1，是广播地址的，由交换机或路由器所拥有，除网络地址和广播地址以外的其他IP地址都可以作为网络设备的IP地址。

网络地址

网络位	主机位
192.168.1	.0
11000000.10101000.00000001	.00000000

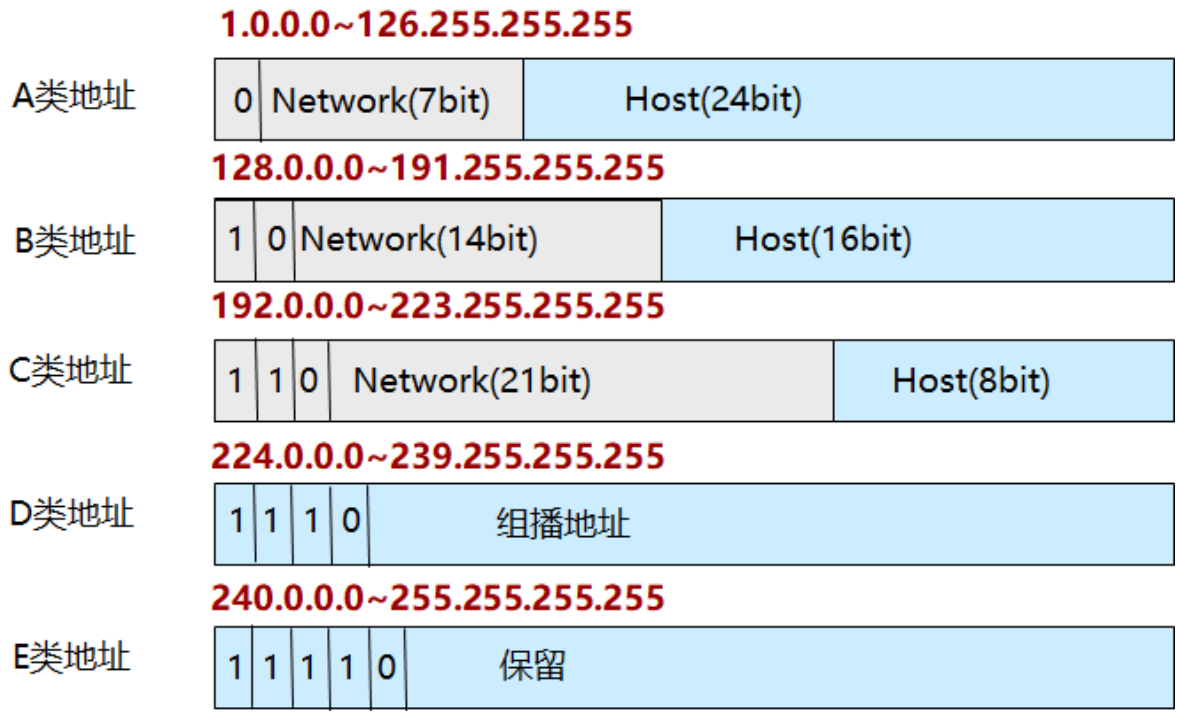
广播地址

网络位	主机位
192.168.1	.255
11000000.10101000.00000001	.11111111

知乎 @吴某某

3、IPv4地址的分类

官方制定的IP地址分配方案，早期设备性能不足，这种IP地址的分类方案可以提高设备转发效率。



A类地址：前8位为网络地址，后24位为主机地址，网络位第一位必须是0，因此该类IP地址中网络ID的长度为8位，主机ID的长度为24位，该类IP地址范围为1.0.0.0~126.255.255.255，一般用于广域网。

B类地址：前16位为网络地址，后16位为主机地址。网络位的前2为必须是10，因为该类IP地址中网络ID的长度为16位，主机ID的长度为16位，该类IP地址范围为128.0.0.0~191.255.255.255，一般用于城域网。

C类地址：前24位为网络地址，后8位主机地址。网络位的前3位必须是110，因此该类IP地址中网络ID的长度为24位，主机长度为8位。该类IP地址范围为192.0.0.0~223.255.255.255 ，一般用于局域网。

D类地址：该类IP地址的第一个字节以1110开始，它是一个专门保留的地址，并不指向特定的网络。目前这类IP地址被用在组播中，其地址范围为224.0.0.0~239.255.255.255。

E类地址：该类IP地址以11110开始，为保留地址。其地址范围为240.0.0.0~255.255.255.254

4、特殊的IPv4地址

网络部分	主机部分	地址类型	用途
Any	全“0”	网络地址	代表一个网段
Any	全“1”	广播地址	特定网段的所有节点
127	Any	环回地址	环回测试
全“0”		所有网络	华为VRP路由器用于指定默认路由
全“1”		广播地址	本网段所有节点

知乎 @吴某某

5、公网IPv4地址与私网IPv4地址

公网指是互联网，公网地址就是能在互联网进行路由（被其它设备访问）的IPv4地址，私网地址不能在公网进行路由的IPv4地址。

现在，世界上所有终端系统和网络设备需要的IP地址总数已经超过了32位IPv4地址所能支持的最大地址数4,294,967,296，为节省IPv4地址官方把IPv4中的部分IP地址保留用作私网地址，A、B、C类地址段中都预留了特定范围的地址作为私网地址，为主机分配私网地址节省了公网地址，可以用来缓解IP地址短缺的问题。

企业网络中普遍使用私网地址，不同企业网络中的私网地址可以重叠，默认情况下，网络中的主机无法使用私网地址与公网通信，我们能够上网靠的是ISP组织分给我们的公网IP地址，但是这种IP地址一般不是给个人的，一般都是给一个单位，一个区域的（共享宽带）。

也就是说我们实际上能接触到的一般都是私网地址，即我们用ipconfig命令查到的都是私有地址，也就相当于局域网内的IP地址，当我们真正联网时，会先把数据发送到路由器，然后再由路由器进行处理实现真正的联网操作，路由器的地址才是真正联网的IP地址，也就是公网IP，而我们在自己电脑上查到的都是私网IP。

6、子网掩码

与IPv4地址的格式相同，但不同的是它的网络位是全1，主机位是全0，主要用在私网中使用，IPv4地址和子网掩码一起可以用来唯一的标识一个网段中的某台网络设备。

类别	子网掩码的二进制数值	子网掩码的十进制数值
A	11111111 00000000 00000000 00000000	255.0.0.0
B	11111111 11111111 00000000 00000000	255.255.0.0
C	11111111 11111111 11111111 00000000	255.255.255.0

子网掩码告知路由器，地址的哪一部分是网络地址，哪一部分是主机地址，使路由器正确判断任意IP地址是否是本网段的，从而正确地进行路由。

子网掩码＆ip地址 = 网络地址，如果两人个ip地址的网络地址相同，那么说明它们在同一个子网内，就可以直接通信而不需要路由器，反之如果如果两人个ip地址的网络地址不同，就需要使用到路由器跨网络通信，不同子网中的IPv4地址可以相同。

子网掩码是在IPv4地址资源紧缺的背景下为了解决IP地址分配而产生的虚拟IP技术，通过子网掩码将A、B、C三类地址划分为若干子网，从而显著提高了IP地址的分配效率，有效解决了IP地址资源紧张的局面。另一方面，在企业内网中为了更好地管理网络，网管人员也利用子网掩码的作用，人为地将一个较大的企业内部网络划分为更多个小规模的子网，再利用三层交换机的路由功能实现子网互联，从而有效解决了网络广播风暴和网络病毒等诸多网络管理方面的问题。

7、网关地址

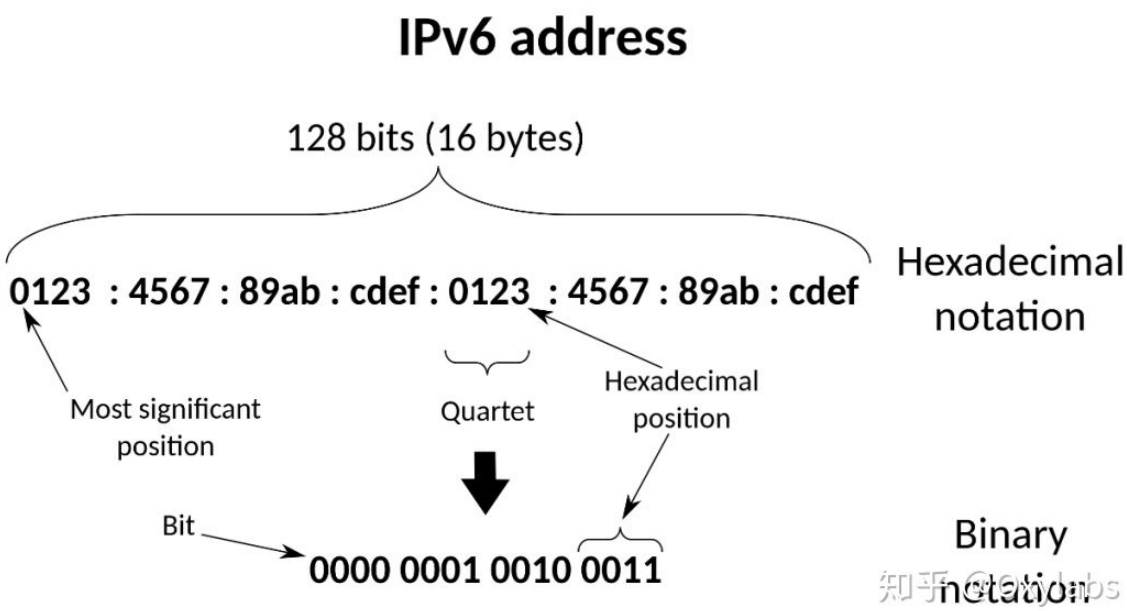
负责私网中出口的计算机，计算机会把要发送的数据优先发送网关地址，由它负责转发，一般由路由器担任（路由器就是一台具有路由功能的计算机）。

8、IPv6地址

IPv4地址组合的数量是有限的，总体而言，可以算出40亿（256 4）个唯一地址。在IPv4地址才开始时，这个数字似乎永远不会过期。但是，现在情况有所不同了。2011年，全球互联网编号分配机构（IANA）分发了IPv4地址空间的最后一块。2015年，IANA正式宣布美国已用完IPv4地址。

直到今天，IPv4地址仍然承载着最多（超过90%）的互联网流量。到目前为止，即使目前存在IPv4地址耗尽的问题，也有一些方法可以继续使用IPv4地址。例如，当仅需要一个唯一的IP地址来代表一组设备时，网络地址转换（NAT）是一种方法。除此之外，IP地址可以重复使用。

当然，我们已经有了彻底耗尽的解决方案-IPv6地址，IPv6地址使用以冒号分隔的十六进制数字。它分为八个16位块，构成一个128位地址方案。



仔细观察，您会发现IPv6地址并不是一种全新的技术。它是Internet协议的最新版本，但它是在1998年开发的，旨在替换IPv4地址。

9、IPv4与IPv6的区别

IPv4	IPv6
32-bit number (2^{32})	128-bit number (2^{128})
Address space: around 4 billion	Address space: 340 trillion, trillion, trillion
e.g. 202.12.29.142	e.g. FE38:DCE3:124C:C1A2:BA03:6745:EF1C:683D
4 groups of numbers, 8 bits per group	16 bits per group, expression compressed
256 combinations	65,536 combinations

知乎 @Oxylabs

地址类型：

IPv4具有三种不同类型的地址：多播，广播和单播。IPv6还具有三种不同类型的地址：任意广播，单播和多播。

数据包大小：

对于IPv4，最小数据包大小为576字节。对于IPv6，最小数据包大小为1208字节。

header区域字段数：

IPv4具有12个标头字段，而IPv6支持8个标头字段。

可选字段：

IPv4具有可选字段，而IPv6没有。但是，IPv6具有扩展header，可以在将来扩展协议而不会影响主包结构。

配置：

在IPv4中，新装的系统必须配置好才能与其他系统通信。在IPv6中，配置是可选的，它允许根据所需功能进行选择。

安全性：

在IPv4中，安全性主要取决于网站和应用程序。它不是针对安全性而开发的IP协议。而IPv6集成了Internet协议安全标准（IPSec）。IPv6的网络安全不像IPv4是可选项，IPv6里的网络安全项是强制性的。

与移动设备的兼容性：

IPv4不适合移动网络，因为正如我们前面提到的，它使用点分十进制表示法，而IPv6使用冒号，是移动设备的更好选择。

主要功能：

IPv6允许直接寻址，因为存在大量可能的地址。但是，IPv4已经广泛传播并得到许多设备的支持，这使其更易于使用。

10、IPv4或IPv6使用哪个

对于使用IPv6还是IPv4这个问题，没有标准答案。在考虑未来的网络体验时，IPv6地址就显得至关重要。即使在我们已经没有网络地址的情况下仍然可以有其他办法使用IPv4地址，但是这些选项也可能会轻微影响到网络速度或引起其他问题。不过，使用IPv6需要开发支持IPv6的新技术和产品。IPv6的速度显然不比IPv4快，但是从IPv4完全更改为IPv6将为Internet提供更大的唯一IP池。那么为什么我们仍在使用IPv4？

问题就在于IPv4和IPv6无法相互通信。这就是为什么IPv6的集成和适配很复杂。大多数网站或应用程序仅支持IPv4类型的IP地址。想象一下突然更改每个设备的IP地址。用户将无法访问大多数网站或应用程序，而我们在互联网上将陷入一片混乱。从旧的IP类型转换为新的IP类型的过程应分步完成。例如，这两个协议能够并行运行。此功能称为双重堆栈。它允许用户同时访问IPv4和IPv6内容。

三、TCP协议与UDP协议

1、TCP/IP、TCP、UDP是什么

TCP/IP协议是一个协议簇，里面包括很多协议的，UDP只是其中的一个， 之所以命名为TCP/IP协议，因为TCP、IP协议是两个很重要的协议，就用他两命名了，而TCP和UDP是位于TCP/IP模型中传输层的两个协议，他们代表着TCP/IP模型所具备的两种通信模式。

TCP (Transmission Control Protocol）传输控制协议，也叫数据流协议 。

UDP (User Datagram Protocol）用户数据报文协议，也叫报文协议。

2、TCP协议介绍

TCP是一种面向广域网的通信协议，目的是在跨越多个网络通信时，为两个通信端点之间提供一条安全可靠的通信方式：

TCP的主要特点：

- 基于流的方式
- 面向连接；
- 可靠通信方式；
- 在网络状况不佳的时候尽量降低系统由于重传带来的带宽开销；
- 通信连接维护是面向通信的两个端点的，而不考虑中间网段和节点。

TCP的传输机制：

数据分片：在发送端对用户数据进行分片，在接收端进行重组，由TCP确定分片的大小并控制分片和重组；

到达确认：接收端接收到分片数据时，根据分片数据序号向发送端发送一个确认；

超时重发：发送方在发送分片时启动超时定时器，如果在定时器超时之后没有收到相应的确认，重发分片；

滑动窗口：TCP连接每一方的接收缓冲空间大小都固定，接收端只允许另一端发送接收端缓冲区所能接纳的数据，TCP在滑动窗口的基础上提供流量控制，防止较快主机致使较慢主机的缓冲区溢出；

失序处理：作为IP数据报来传输的TCP分片到达时可能会失序，TCP将对收到的数据进行重新排序，将收到的数据以正确的顺序交给应用层；

重复处理：作为IP数据报来传输的TCP分片会发生重复，TCP的接收端必须丢弃重复的数据；

数据校验：TCP将保持它首部和数据的检验和，这是一个端到端的检验和，目的是检测数据在传输过程中的任何变化。如果收到分片的检验和有差错，TCP将丢弃这个分片，并不确认收到此报文段导致对端超时并重发。

TCP的首部格式：

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|          Source Port          |       Destination Port        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                        Sequence Number                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Acknowledgment Number                      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Data |           |U|A|P|R|S|F|                               |
| Offset| Reserved  |R|C|S|S|Y|I|            Window             |
|       |           |G|K|H|T|N|N|                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|           Checksum            |         Urgent Pointer        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Options                    |    Padding    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                             data                              |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

```
Source Port 是源端口，16位。
Destination Port是目的端口，16位。
Sequence Number是发送数据包中的第一个字节的序列号，32位。
Acknowledgment Number是确认序列号，32位。
Data Offset是数据偏移，4位，该字段的值是TCP首部（包括选项）长度除以4。
```

```
标志位： 6位
    URG表示Urgent Pointer字段有意义：
    ACK表示Acknowledgment Number字段有意义
    PSH表示Push功能，RST表示复位TCP连接
    SYN表示SYN报文（在建立TCP连接的时候使用）
    FIN表示没有数据需要发送了（在关闭TCP连接的时候使用）
    Window表示接收缓冲区的空闲空间，16位，用来告诉TCP连接对端自己能够接收的最大数据长度。
Checksum是校验和，16位。
Urgent Pointers是紧急指针，16位，只有URG标志位被设置时该字段才有意义，表示紧急数据相对序列号（Sequence Number字段的值）的偏移。
```

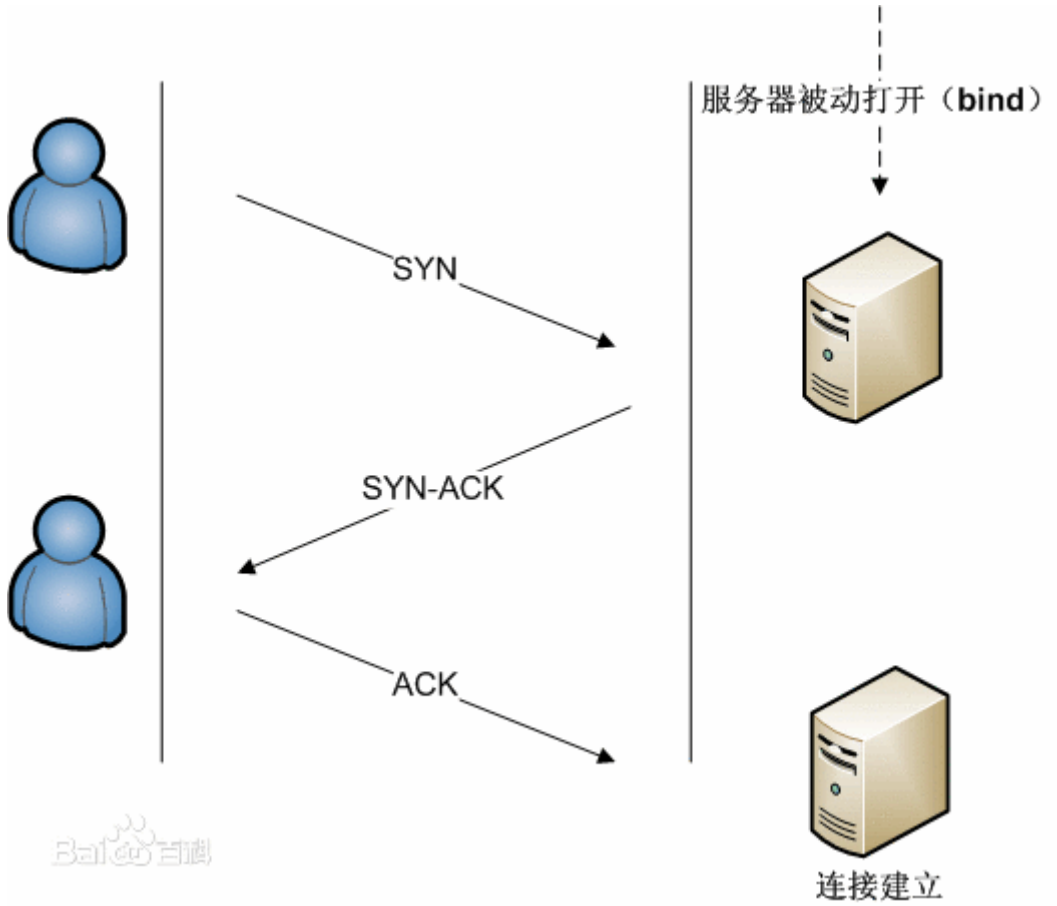
3、TCP的连接过程

TCP是因特网中的传输层协议，使用三次握手协议建立连接。当主动方发出SYN连接请求后，等待对方回答SYN+ACK，并最终对对方的 SYN 执行 ACK 确认。这种建立连接的方法可以防止产生错误的连接，TCP使用的流量控制协议是可变大小的滑动窗口协议。

建立一个连接需要三次握手，而终止一个连接要经过四次握手，这是由TCP的半关闭（half-close）造成的。

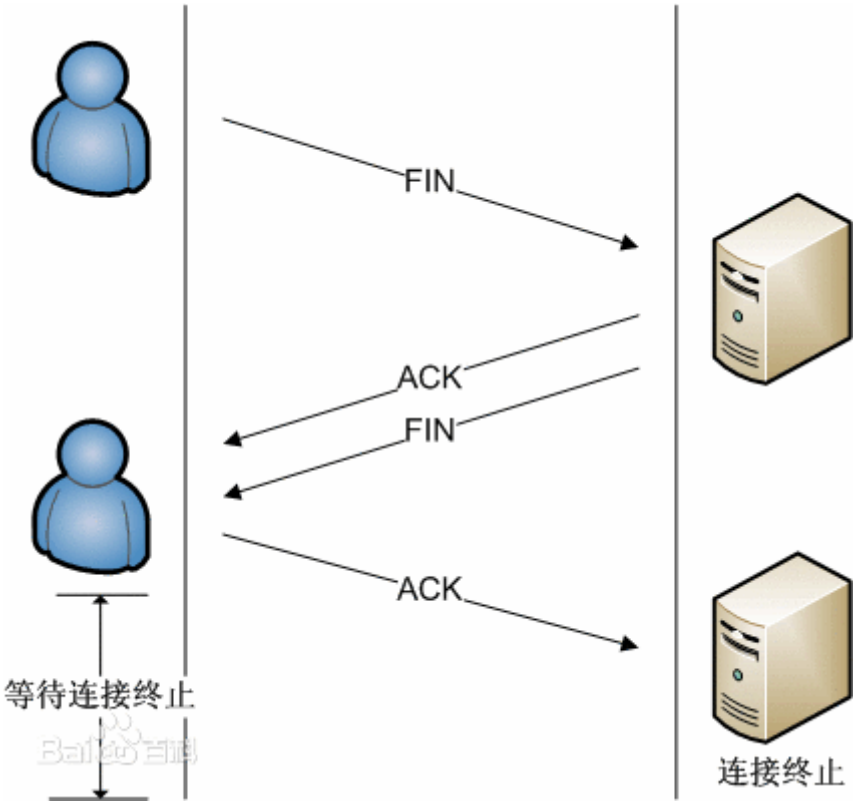
TCP三次握手的过程：

1. 客户端发送SYN（SEQ=x）报文给服务器端，进入SYN_SEND状态。
2. 服务器端收到SYN报文，回应一个SYN （SEQ=y）ACK（ACK=x+1）报文，进入SYN_RECV状态。
3. 客户端收到服务器端的SYN报文，回应一个ACK（ACK=y+1）报文，进入Established状态。
   三次握手完成，TCP客户端和服务器端成功地建立连接，可以开始传输数据了。



TCP四次挥手的过程：

1. 某个应用进程首先调用close，称该端执行“主动关闭”（active close）。该端的TCP于是发送一个FIN分节，表示数据发送完毕。
2. 接收到这个FIN的对端执行 “被动关闭”（passive close），这个FIN由TCP确认。
   注意：FIN的接收也作为一个文件结束符（end-of-file）传递给接收端应用进程，放在已排队等候该应用进程接收的任何其他数据之后，因为，FIN的接收意味着接收端应用进程在相应连接上再无额外数据可接收。
3. 一段时间后，接收到这个文件结束符的应用进程将调用close关闭它的套接字。这导致它的TCP也发送一个FIN。
4. 接收这个最终FIN的原发送端TCP（即执行主动关闭的那一端）确认这个FIN。 [3]
   既然每个方向都需要一个FIN和一个ACK，因此通常需要4个分节，所以叫四次挥手。



注意：

1. “通常”是指，某些情况下，步骤1的FIN随数据一起发送，另外，步骤2和步骤3发送的分节都出自执行被动关闭那一端，有可能被合并成一个分节。
2. 在步骤2与步骤3之间，从执行被动关闭一端到执行主动关闭一端流动数据是可能的，这称为“半关闭”（half-close）。
3. 当一个Unix进程无论自愿地（调用exit或从main函数返回）还是非自愿地（收到一个终止本进程的信号）终止时，所有打开的描述符都被关闭，这也导致仍然打开的任何TCP连接上也发出一个FIN。
4. 无论是客户还是服务器，任何一端都可以执行主动关闭。通常情况是，客户执行主动关闭，但是某些协议，例如，HTTP/1.0却由服务器执行主动关闭。

4、UDP协议介绍

是一种支持无连接的传输协议，UDP协议与TCP协议一样用于处理数据包，在OSI模型中，两者都位于传输层，处于IP协议的上一层，与TCP互为补充，UDP为应用程序提供了一种无需建立连接就可以发送封装的 IP 数据包的方法。

由于传输数据不建立连接，因此也就不需要维护连接状态，包括收发状态等，因此一台服务机可同时向多个客户机传输相同的消息（多对多通信）。

UDP的主要特点：

UDP它除了给应用程序发送数据包功能并允许它们在所需的层次上架构自己的协议之外，几乎没有做什么特别的事情，而TCP协议几乎做了所有通信协议该做的事情。

UDP不提供数据包分组、组装和不能对数据包进行排序的缺点，也就是说，当报文发送之后，是无法得知其是否安全完整到达的。

UDP的首部格式：

UDP信息包的标题很短，只有8个字节，相对于TCP的20个字节信息包而言UDP的额外开销很小。

```
 0      7 8     15 16    23 24    31
+--------+--------+--------+--------+
|     Source      |   Destination   |
|      Port       |      Port       |
+--------+--------+--------+--------+
|                 |                 |
|     Length      |    Checksum     |
+--------+--------+--------+--------+
|
|          data octets ...
+---------------- ...
```

User Datagram Header Format

Length：

数据报的长度是指包括报头和数据部分在内的总字节数。因为报头的长度是固定的，所以该域主要被用来计算可变长度的数据部分（又称为数据负载）。

数据报的最大长度根据操作环境的不同而各异。从理论上说，包含报头在内的数据报的最大长度为65535字节。不过，一些实际应用往往会限制数据报的大小，有时会降低到8192字节。

吞吐量不受拥挤控制算法的调节，只受应用软件生成数据的速率、传输带宽、源端和终端主机性能的限制。

UDP是面向报文的。发送方的UDP对应用程序交下来的报文，在添加首部后就向下交付给IP层，既不拆分，也不合并，而是保留这些报文的边界，由应用程序需要选择合适的报文大小。

Checksum：

UDP协议使用报头中的校验值来保证数据的安全。校验值首先在数据发送方通过特殊的算法计算得出，在传递到接收方之后，还需要再重新计算。

如果某个数据报在传输过程中被第三方篡改或者由于线路噪音等原因受到损坏，发送和接收方的校验计算值将不会相符，由此UDP协议可以检测是否出错。这与TCP协议是不同的，后者要求必须具有校验值。

UDP的适用范围：

它主要用于不要求分组顺序到达的传输中，分组传输顺序的检查与排序由应用层完成，也就是可靠性由应用层负责，UDP协议适用运行在同一台设备上的多个应用程序进行通信，即使进行网络通信也只适合于一次传输少量数据，使用UDP协议包括：TFTP、SNMP、NFS、DNS、BOOTP。

UDP报文没有可靠性保证、顺序保证和流量控制字段等，可靠性较差。但是正因为UDP协议的控制选项较少，在数据传输过程中延迟小、数据传输效率高，适合对可靠性要求不高的应用程序，或者可以保障可靠性的应用程序。

多媒体数据流，不产生任何额外的数据，即使知道有破坏的包也不进行重发。当强调传输性能而不是传输的完整性时，如：音频和多媒体应用，UDP是最好的选择。在数据传输时间很短，以至于此前的连接过程成为整个流量主体的情况下，UDP也是一个好的选择。

5、选择TCP还是UDP

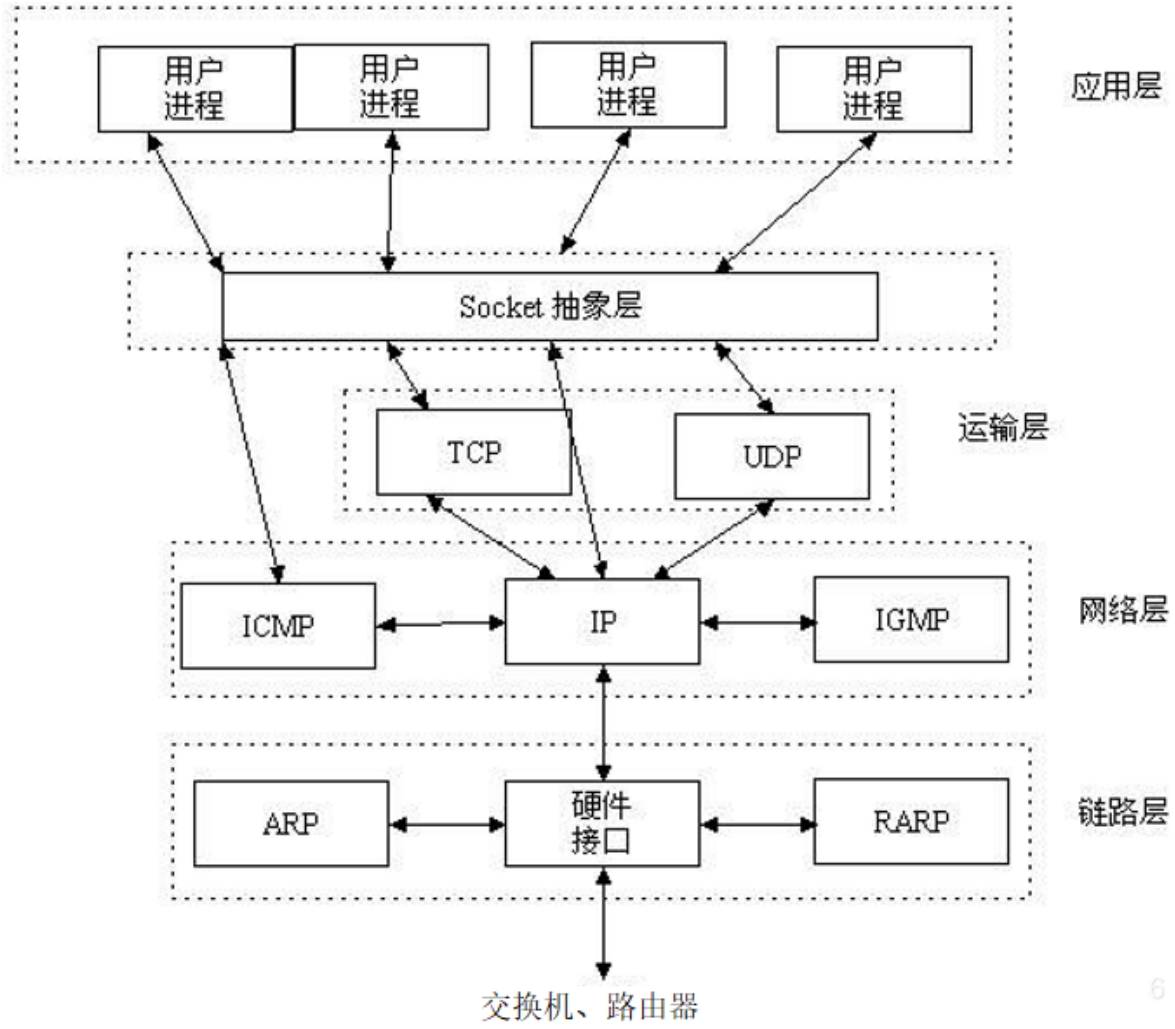
TCP向上层提供面向连接的可靠服务，UDP向上层提供无连接不可靠服务。
虽然 UDP 并没有 TCP 传输来的准确，但是 也能在很多实时性要求高的地方有所作为。

	UDP	TCP
是否连接	无连接	面向连接
是否可靠	不可靠传输，不使用流量控制和拥塞控制	可靠传输，使用流量控制和拥塞控制
连接对象个数	支持一对一，一对多，多对一和多对多交互通信	只能是一对一通信
传输方式	面向报文	面向字节流
首部开销	首部开销小，仅8字节	首部最小20字节，最大60字节
适用场景	适用于实时应用（IP电话、视频会议、直播等）	适用于要求可靠传输的应用

四、套接字

1、什么是套接字

套接字（socket）是一种通信机制，是通信的两方的一种约定，socket屏蔽了各个协议的通信细节，对用户进程提供了一套可以统一、方便的使用TCP/IP协议的接口。这使得程序员无需关注协议本身，直接使用socket提供的接口与不同主机间的进程互联通信。



2、创建套接字

套接字描述符类似于文件描述符，UNIX、Linux把网络当文件看待，发送数据即写文件write，接收数据即读文件read，销毁socket对象即关闭文件close，在UNIX、Linux系统下一切皆文件。

```
#include <sys/socket.h>

int socket (int domain, int type, int protocol);
功能：在内核中创建一个套接字对象
domain：通信地址类型
    AF_UNIX/AF_LOCAL/AF_FILE: 本地通信(进程间通信)
    AF_INET: 基于IPv4(32位IP地址)的网络通信
    AF_INET6: 基于IPv6(128位IP地址)的网络通信
type：通信协议类型
    SOCK_STREAM: 数据流协议，即TCP协议
    SOCK_DGRAM: 数据报协议，即UDP协议
protocol：特别通信协议，一般不用，置0即可
返回值：成功返回套接字描述符，失败返回-1
```

3、套接字的连接

当socket对象创建好以后，它并不能立即被其它socket对象连接并通信，需要把一个地址与Socket对象进行绑定，这样它才能被其它socket对象连接并通信。

```
// 基本地址类型，它是socket系列接口的表面参数，而实际使用的是sockaddr_un或sockaddr_in
struct sockaddr
```

```
{
    sa_family_t sa_family;      // 地址类型，与创建socket对象时的domain参数一至即可
    char      sa_data[14];      // 它是只是占位而已
};

// 本地地址类型
#include <sys/un.h>
struct sockaddr_un
{
    sa_family_t sun_family;
    char        sun_path[];     // 套接字文件路径
};

// 网络地址类型
#include <netinet/in.h>
struct sockaddr_in
{
    sa_family_t sin_family;
    in_port_t sin_port;         // 端口号，用于区分通信方的进程，1024~65535
    struct in_addr sin_addr;    // IP地址
};

struct in_addr
{
    in_addr_t s_addr;           // 32位IPv4地址
};
typedef uint32_t in_addr_t;

#include <sys/socket.h>
int bind (int sockfd, const struct sockaddr* addr,socklen_t addrlen);
功能：将套接字和通信地址绑定在一起，由被动连接的Socket对象调用
addr：sockaddr_un、sockaddr_in结构变量的地址
addrlen：地址结构变量的字节数，便于bind区分用户提供的是sockaddr_un或sockaddr_in
返回值：成功返回0，失败返回-1


#include <sys/socket.h>
int connect (int sockfd, const struct sockaddr* addr,socklen_t addrlen);
功能：socket对象A连接socket对象B，由主动连接的Socket对象调用
sockfd：socket对象A的描述符，也就是连接的发起者
addr：socket对象B的通信地址，socket对象B必须与该通信地址绑定过
addrlen：地址结构变量的字节数
返回值：成功返回0，失败返回-1
```

五、IP地址转换与字节序转换

主机字节序因处理器架构而不同，有的采用小端字节序，有的采用大端字节序，网络字节序则固定采用大端字节序，所以在网络通信时需要把数据类型大于1字节的数据转换成网络字节序。

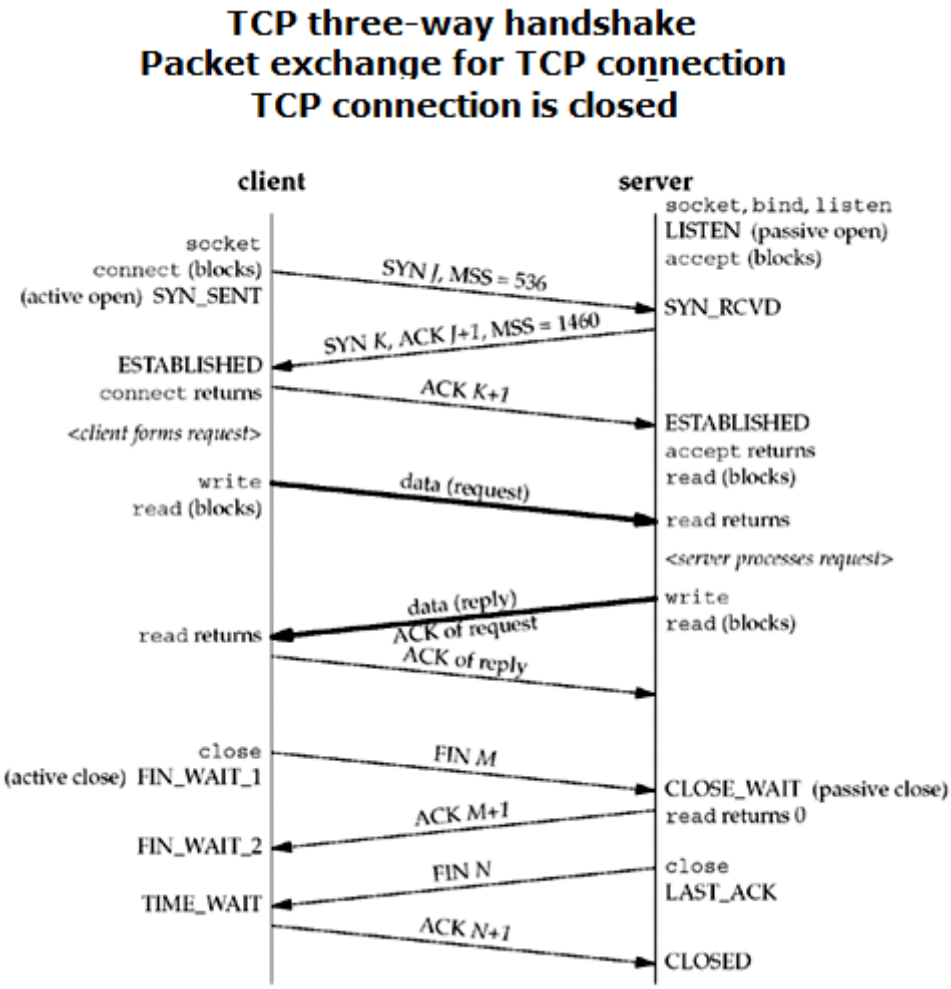
```c
#include <arpa/inet.h>

// 32位无符号整数，主机字节序 -> 网络字节序
uint32_t htonl (uint32_t hostlong);

// 16位无符号整数，主机字节序 -> 网络字节序
uint16_t htons (uint16_t hostshort);

// 32位无符号整数，网络字节序 -> 主机字节序
uint32_t ntohl (uint32_t netlong);

// 16位无符号整数，网络字节序 -> 主机字节序
uint16_t ntohs (uint16_t netshort);
```

日常生活中使用IPv4地址是点分十进制格式，而在程序中使用的是32位4字节无符号整数，并且还要是采用大端字节序。

```c
#include <arpa/inet.h>

// 点分十进制字符串 -> 网络字节序32位无符号整数
in_addr_t inet_addr (const char* cp);

// 点分十进制字符串 -> 网络字节序32位无符号整数
int inet_aton (const char* cp, struct in_addr* inp);

// 网络字节序32位无符号整数 -> 点分十进制字符串
char* inet_ntoa (struct in_addr in);
```

六、基于TCP协议的网络Socket通信

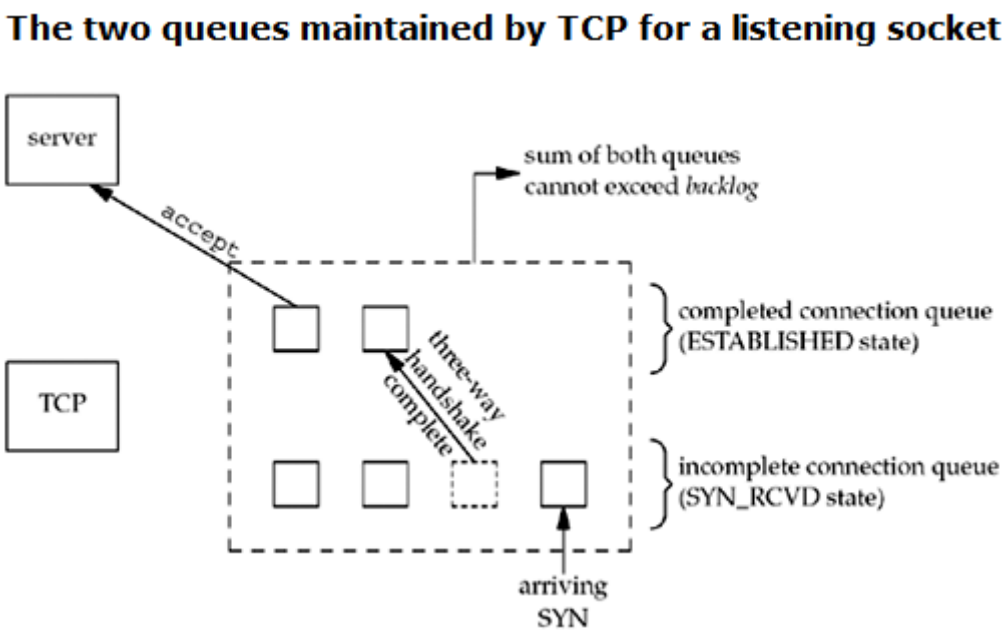
由于TCP协议是基于面向连接的，而且在连接还需要进行三次握手，所以在通信前被连接Scoket对象还需要开启监听和等待连接的步骤。



1、开启监听

在TCP服务器编程中listen函数使主动连接套接字变为被动连接套接口，使得一个进程可以接受其它进程的连接请求，从而成为一个服务器进程。

```
#include <sys/socket.h>
int listen (int sockfd, int backlog);
sockfd: 一个已绑定未被连接的套接字描述符
backlog：连接请求队列(queue of pending connections)的最大长度（一般由2到4）
    用SOMAXCONN则为系统给出的最大值，TCP为侦听套接字维护的两个队列
```



2、等待连接

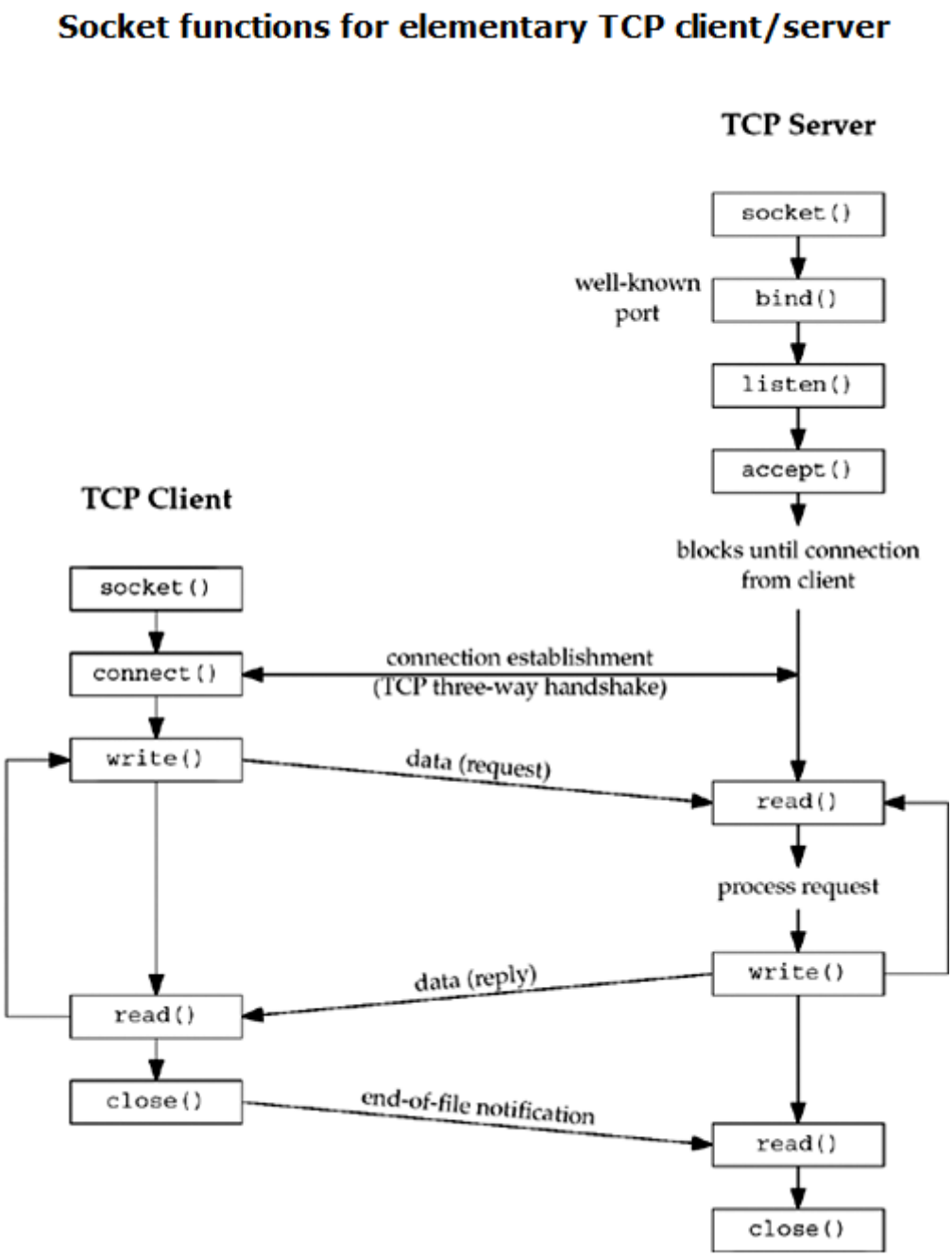
```
int accept (int sockfd, struct sockaddr* addr,socklen_t* addrlen);
功能：从sockfd参数所标识套接字的未决连接请求队列中，提取第一个连接请求。
    同时创建一个新的套接字，用于在该连接中通信，返回该套接字的描述符。
addr：用于输出连接请求发起者的地址信息
addrlen：既是输入也是输出
    输入：它负责告诉accept函数addr的容量
    输出：它能从accept获取连接者的addr的字节数
返回值：成功返回通信套接字描述符，失败返回-1。
```

3、TCP的数据发送和接收

Socket对象的本质就是内核文件对象，所以发送数据可以write函数，接收数据可以使用read函数，关闭Socket对象close，但Socket在write、read的基础上封装了自己发送数据、接收数据的函数。

```
ssize_t recv (int sockfd, void* buf, size_t len,int flags);
功能：通过sockfd参数所标识的套接字，期望接收len个字节到buf所指向的缓冲区中。
返回值：成功返回实际接收到的字节数，失败返回-1。
flags：暂时写0即可，后续详细介绍

ssize_t send (int sockfd, const void* buf,size_t len, int flags);
功能：通过sockfd参数所标识的套接字，从buf所指向的缓冲区中发送len个字节。
返回值：成功返回实际被发送的字节数，失败返回-1。
```

3、详细编程步骤



步骤	服务器		客户机		步骤
1	创建套接字	socket	socket	创建套接字	1
2	准备地址	...	...	准备地址	2
3	绑定套接字	bind		----	
4	监听套接字	listen		----	
5	接受连接	accept	connect	建立链接	3
6	接收请求	recv	send	发送请求	4
7	发送响应	send	recv	接收响应	5
8	关闭套接字	close	close	关闭套接字	6

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
#include <unistd.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

void error_exit(const char *func)
{
    printf("%s:%m\n", func);
    exit(EXIT_FAILURE);
}

void *run(void *arg)
{
    int cli_fd = *(int *)arg;
    char buf[4096];
    for (;;)
    {
        // 接收数据
        size_t ret = recv(cli_fd, buf, sizeof(buf), 0);
        if (0 >= ret || 0 == strcmp("quit", buf))
            break;

        printf("recv:%s bytes:%u\n", buf, ret);
        strcat(buf, ":return");

        // 返回数据
        ret = send(cli_fd, buf, strlen(buf) + 1, 0);
        if (0 >= ret)
            break;
    }
    printf("通信结束！\n");
    // 关闭通信socket
    close(cli_fd);
}

int main(int argc, const char *argv[])
{
    if (2 != argc)
    {
        printf("Use:./a.out <port>\n");
        return EXIT_FAILURE;
    }
    // 创建socket对象
    int svr_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (0 > svr_fd)
        error_exit("socket");

    // 准备通信地址
    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(atoi(argv[1]));
    addr.sin_addr.s_addr = inet_addr("10.0.2.15");
    socklen_t addrlen = sizeof(addr);

    // 绑定socket对象和通信地址
    if (bind(svr_fd, (struct sockaddr *)&addr, addrlen))
```

```
        error_exit("bind");

    // 开启监听
    if (listen(svr_fd, 3))
        error_exit("listen");

    for (;;)
    {
        // 等待连接
        int cli_fd = accept(svr_fd, NULL, NULL);
        if (0 > cli_fd)
            error_exit("accept");

        pthread_t tid;
        pthread_create(&tid, NULL, run, &cli_fd);
        usleep(1000);
    }

    // 关闭连接socket
    close(svr_fd);
    return EXIT_SUCCESS;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

void error_exit(const char* func)
{
    printf("%s:%m\n",func);
    exit(EXIT_FAILURE);
}

int main(int argc,const char* argv[])
{
    // 创建socket对象
    int cli_fd = socket(AF_INET,SOCK_STREAM,0);
    if(0 > cli_fd)
        error_exit("socket");

    // 准备server的通信地址
    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(2123);
    addr.sin_addr.s_addr = inet_addr("10.0.2.15");
    socklen_t addrlen = sizeof(addr);

    // 连接server
    if(connect(cli_fd,(struct sockaddr*)&addr,addrlen))
        error_exit("connect");

    char buf[4096];
```

```c
    for(;;)
    {
        printf(">>>");
        scanf("%s",buf);

        // 发送数据
        size_t ret = send(cli_fd,buf,strlen(buf)+1,0);
        if(0 >= ret || 0 == strcmp("quit",buf))
            break;

        // 接收数据
        ret = recv(cli_fd,buf,sizeof(buf),0);
        if(0 >= ret)
            break;

        printf("recv:%s bytes:%u\n",buf,ret);
    }
    printf("通信结束！\n");
    // 关闭socket对象
    close(cli_fd);
    return EXIT_SUCCESS;
}
```

七、基于UDP协议的网络Socket通信

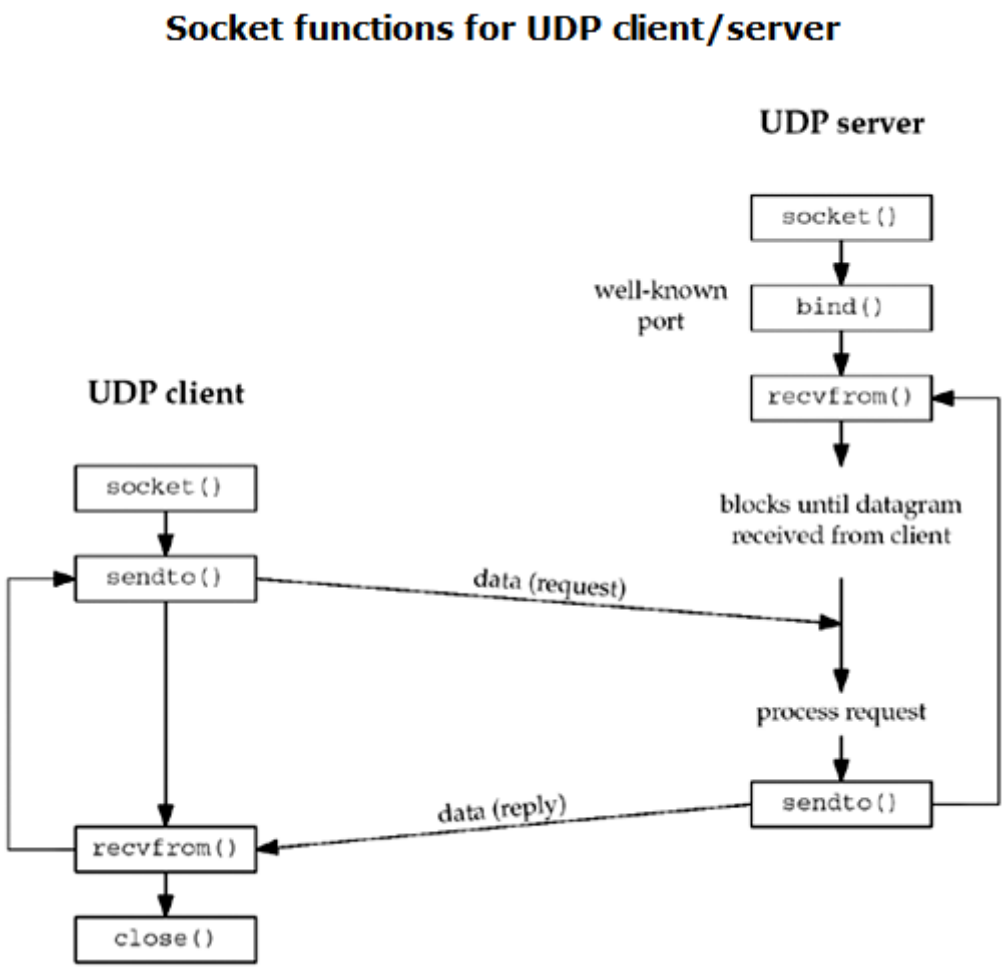
1、UDP协议的特点

- UDP是无连接的，即通信时不需要创建连接，发送数据结束时也没有连接可以释放，所以减小了开销和发送数据前的延时；
- UDP采用最大努力交付，不保证可靠交付，因此主机不需要维护复杂的连接状态；
- UDP是面向报文的，只在应用层交下来的报文前增加了首部后就向下交付IP层，不需要解决粘包的问题。
- UDP是无阻塞控制的，即使网络中存在阻塞，也不会影响发送端的发送频率
- UDP支持一对一、一对多、多对一、多对多的交互通信；
- UDP的首部开销小，只有8个字节，它比TCP的20个字节的首部要短；

2、UDP的数据发送和接收

```c
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
                        const struct sockaddr *dest_addr, socklen_t addrlen);
功能：UDP专用的数据发送函数。
dest_addr：收件人的地址。
addrlen：地址长度。

ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                          struct sockaddr *src_addr, socklen_t *addrlen);
功能：UDP专用的数据接收函数。
src_addr：发件人的地址，也是数据返回时的地址
addrlen：地址长度。
```

3、详细编程步骤



步骤	服务器		客户机		步骤
1	创建套接字	socket	socket	创建套接字	1
2	准备地址	...	...	准备地址	2
3	绑定套接字	bind		----	
4	接收请求	recvfrom	sendto	发送请求	3
5	发送响应	sendto	recvfrom	接收响应	4
6	关闭套接字	close	close	关闭套接字	5

```c
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, const char *argv[])
{
    // 创建socket对象
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (0 > sockfd)
    {
        perror("socket");
        return -1;
```

```c
    }

    // 准备通信地址(本机的)
    struct sockaddr_in addr = {}, src_addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5566);
    addr.sin_addr.s_addr = inet_addr("192.168.56.103");
    socklen_t addrlen = sizeof(addr);

    // 绑定地址与socket对象
    if (bind(sockfd, (struct sockaddr *)&addr, addrlen))
    {
        perror("bind");
        return -1;
    }

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for (;;)
    {
        // 接收数据
        int size = recvfrom(sockfd, buf, buf_size, 0, (struct sockaddr *)&src_addr, &addrlen);
        buf[size] = '\0';
        printf("from:%s recv:%s byte:%d\n", inet_ntoa(src_addr.sin_addr), buf, size);
        strcat(buf, ":return");

        // 返回数据
        sendto(sockfd, buf, strlen(buf) + 1, 0, (struct sockaddr *)&src_addr, addrlen);
    }

    return 0;
}

```

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, const char *argv[])
{
    // 创建socket对象
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    // 准备通信地址(目标的)
```

```c
    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5566);
    addr.sin_addr.s_addr = inet_addr("192.168.56.103");
    socklen_t addrlen = sizeof(addr);

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for (;;)
    {
        printf(">>>");
        scanf("%s", buf);
        if (0 == strcmp("quit", buf))
        {
            printf("通信结束！");
            close(sockfd);
            break;
        }

        sendto(sockfd, buf, strlen(buf)+1, 0, (struct sockaddr *)&addr, addrlen);
        int size = recvfrom(sockfd, buf, buf_size, 0, (struct sockaddr *)&addr, &addrlen);
        printf("from:%s recv:%s byte:%d\n", inet_ntoa(addr.sin_addr), buf, size);
    }
}
```

4、UDP的连接操作

UPD协议底层是否需要连接操作，客户端但可以在Socket层面进行连接，连接后的Socket对象在后续的通信过程中就不再需要通信地址了。

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc,const char* argv[])
{
    // 创建socket对象
    int sockfd = socket(AF_INET,SOCK_DGRAM,0);
    if(0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    // 准备通信地址(目标的)
    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(7777);
    addr.sin_addr.s_addr = inet_addr("47.97.229.46");
```

```
    socklen_t addrlen = sizeof(addr);

    // 连接服务器
    if(connect(sockfd,(struct sockaddr*)&addr,addrlen))
    {
        perror("connect");
        return -1;
    }


    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        printf(">>>");
        scanf("%s",buf);

        send(sockfd,buf,strlen(buf)+1,0);
        if(0 == strcmp("quit",buf))
        {
            printf("通信结束！");
            close(sockfd);
            break;
        }

        int size = recv(sockfd,buf,buf_size,0);
        printf("from:%s recv:%s byte:%d\n",inet_ntoa(addr.sin_addr),buf,size);
    }
}
```

八、本地Socket通信

1、本地Socket通信与网络Socket通信的区别

网络Socket通信是把网卡抽象成Socket文件配合TCP/IP协议簇，能够使当前进程与其它计算机的进程进行网络通信。

本地Socket通信是在文件系统中创建Socket文件，能够使当前进程与本机的其它进程进行通信（IPC进程间通信）。

2、创建Socket文件

使用sockaddr_un类型的通信地址，当调用socket对象与通信地址绑定时，会自动创建socket文件。

```
// 本地地址类型
#include <sys/un.h>
struct sockaddr_un
{
    sa_family_t sun_family;
    char        sun_path[];     // 套接字文件路径
};

#include <sys/socket.h>
int bind (int sockfd, const struct sockaddr* addr,socklen_t addrlen);
```

3、基于TCP协议的本地Socket通信

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

int main(int argc,const char* argv[])
{
    int sockfd = socket(AF_LOCAL,SOCK_STREAM,0);
    if(0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    struct sockaddr_un addr = {};
    addr.sun_family = AF_LOCAL;
    strcpy(addr.sun_path,"tcp_socket");
    socklen_t addrlen = sizeof(addr);

    if(bind(sockfd,(struct sockaddr*)&addr,addrlen))
    {
        perror("bind");
        return -1;
    }

    if(listen(sockfd,3))
    {
        perror("listen");
        return -1;
    }

    for(;;)
    {
        int clifd = accept(sockfd,NULL,0);
        if(0 > clifd)
        {
            perror("accept");
            sleep(1);
            continue;
        }

        if(fork())
            continue;

        char buf[4096];
        size_t buf_size = sizeof(buf);

        for(;;)
        {
            printf("recv...\n");
            int ret = recv(clifd,buf,buf_size,0);
            if(0 >= ret || 0 == strcmp("quit",buf))
            {
                printf("结束通信！");
                break;
```

```
            }

            buf[ret] = '\0';
            printf("recv:%s byte:%d\n",buf,ret);
            strcat(buf,":return");

            ret = send(clifd,buf,strlen(buf)+1,0);
            if(0 >= ret)
            {
                printf("结束通信！");
                break;
            }
        }

        close(clifd);
        return 0;
    }
    return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

int main(int argc,const char* argv[])
{
    int sockfd = socket(AF_LOCAL,SOCK_STREAM,0);
    if(0 > sockfd)
    {
        perror("sockfd");
        return -1;
    }

    struct sockaddr_un addr = {};
    addr.sun_family = AF_LOCAL;
    strcpy(addr.sun_path,"tcp_socket");
    socklen_t addrlen = sizeof(addr);

    if(connect(sockfd,(struct sockaddr*)&addr,addrlen))
    {
        perror("connect");
        return -1;
    }

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        printf(">>>");
        scanf("%s",buf);

        int ret = send(sockfd,buf,strlen(buf)+1,0);
        if(0 >= ret || 0 == strcmp("quit",buf))
        {
            printf("通信结束！\n");
```

```
            break;
        }

        ret = recv(sockfd,buf,buf_size,0);
        if(0 >= ret)
        {
            printf("通信结束! \n");
            break;
        }

        printf("recv:%s byte:%d\n",buf,ret);
    }

    close(sockfd);
    return 0;
}
```

4、基于UDP协议的网络Socket通信

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

int main(int argc,const char* argv[])
{
    // 创建socket对象
    int sockfd = socket(AF_LOCAL,SOCK_DGRAM,0);
    if(0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    // 准备通信地址(本机的)
    struct sockaddr_un addr = {}, src_addr = {};
    addr.sun_family = AF_LOCAL;
    strcpy(addr.sun_path,"udp_ctos");
    socklen_t addrlen = sizeof(addr);

    // 绑定地址与socket对象
    if(bind(sockfd,(struct sockaddr*)&addr,addrlen))
    {
        perror("bind");
        return -1;
    }

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        // 接收数据
```

```c
        int size = recvfrom(sockfd,buf,buf_size,0,(struct sockaddr*)&src_addr,&addrlen);

        printf("from:%s recv:%s byte:%d\n",src_addr.sun_path,buf,size);
        strcat(buf,":return");

        // 返回数据
        sendto(sockfd,buf,strlen(buf)+1,0,(struct sockaddr*)&src_addr,addrlen);
    }

    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

int main(int argc,const char* argv[])
{
    // 创建socket对象
    int sockfd = socket(AF_LOCAL,SOCK_DGRAM,0);
    if(0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    // 准备通信地址(自己的)
    struct sockaddr_un addr = {}, dest_addr = {};
    addr.sun_family = AF_LOCAL;
    strcpy(addr.sun_path,"udp_stoc");
    socklen_t addrlen = sizeof(addr);

    // 绑定地址与socket对象
    if(bind(sockfd,(struct sockaddr*)&addr,addrlen))
    {
        perror("bind");
        return -1;
    }

    // 目标的
    dest_addr.sun_family = AF_LOCAL;
    strcpy(dest_addr.sun_path,"udp_ctos");

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        printf(">>>");
        scanf("%s",buf);
        if(0 == strcmp("quit",buf))
        {
            printf("通信结束！");
```

```
            close(sockfd);
            break;
        }

        sendto(sockfd,buf,strlen(buf)+1,0,(struct sockaddr*)&dest_addr,addrlen);
        int size = recvfrom(sockfd,buf,buf_size,0,(struct sockaddr*)&dest_addr,&addrlen);
        printf("recv:%s byte:%d\n",buf,size);
    }
}
```

九、Windows系统下的Socket通信

1、Windows系统的Socket介绍

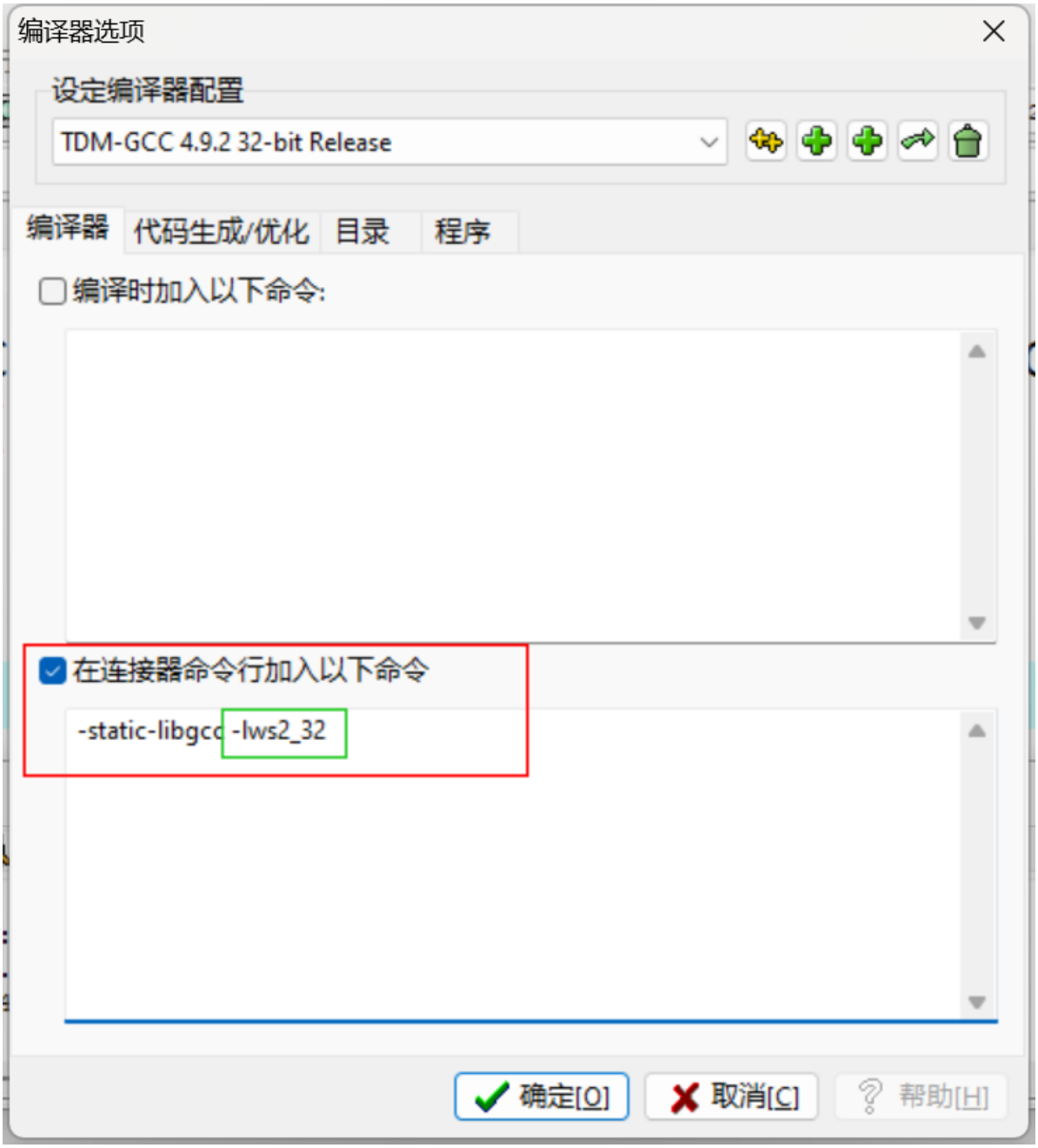
Windows系统的Socket简称Winsock，其中大部分的内容是参考了UNIX系统的Socket设计的，所以很多地方与Linux系统的Socket类似，因此在大多数情况下只需要简单更改Linux环境下编写好的Socket程序代码，就能在Windows系统下运行。

大多数项目的服务端程序都运行在Linux、UNIX系列的操作系统下开发运行的，而多数客户端是在Windows平台下开发运行的，所以学习Socket通信必须兼顾Windows和Linux两大平台。

2、Windows系统的Socket的库和头文件

Windows系统下编写Socket代码只需要包含winsock2.h头文件，但在编译时需要额外链接ws2_32.lib库。

以Dev-C++集成开发环境为例：工具菜单-> 编译选项-> 在链接器命令行加入以下命令。



3、Winsock库的初始化和注销

```
int WSAStartup(WORD wVersionRequested,LPWSADATA lpWSAData);
功能：初始化Winsock库
wVersionRequested：Winsock库的版本信息，用于指定要使用的版本
    高8位为副版本号，低8位为主版本号，个人建议使用2.2的版本，所以就传递0x0202
    可以使用MAKEWORD宏创建版本信息
    MAKEWORD(1,2); // 主版本号为1，副版本号为2
    MAKEWORD(2,2); // 主版本号为2，副版本号为2
lpWSAData：WSADATA结构变量的地址，是个输出型的参数，用于获取初始化后的Winsock库的相关信息。
返回值：成功返回0，失败返回错误码

typedef struct WSAData {
    WORD        wVersion;
    WORD        wHighVersion;
#ifdef _WIN64
    unsigned short  iMaxSockets;
    unsigned short  iMaxUdpDg;
    char        *lpVendorInfo;
```

```
    char        szDescription[WSADESCRIPTION_LEN+1];
    char        szSystemStatus[WSASYS_STATUS_LEN+1];
#else
    char        szDescription[WSADESCRIPTION_LEN+1];
    char        szSystemStatus[WSASYS_STATUS_LEN+1];
    unsigned short  iMaxSockets;
    unsigned short  iMaxUdpDg;
    char        *lpVendorInfo;
#endif
} WSADATA, *LPWSADATA;


int WSACleanup(void);
功能：注销Winsock库
返回值：成功返回0，失败返回错误码
```

```
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>

void error_exit(const char* error)
{
    printf("%s:%m\n",error);
    exit(1);
}

int main()
{
    WSADATA wsaData;
    if(WSAStartup(MAKEWORD(1,2),&wsaData))
        error_exit("WsaStartup");

    printf("%x %x\n",(char)wsaData.wVersion,(char)wsaData.wHighVersion);

    WSACleanup();
}
```

4、Windows系统的Socket相关函数

```
SOCKET socket(int af,int type,int protocol);

int bind(SOCKET s,const struct sockaddr *name,int namelen);

int listen(SOCKET s,int backlog);

SOCKET accept(SOCKET s,struct sockaddr *addr,int *addrlen);

int connect(SOCKET s,const struct sockaddr *name,int namelen);

int closesocket(SOCKET s);

int send(SOCKET s,const char *buf,int len,int flags);

int sendto(SOCKET s,const char *buf,int len,int flags,const struct sockaddr
*to,int tolen);

```

```
int recv(SOCKET s,char *buf,int len,int flags);

int recvfrom(SOCKET s,char *buf,int len,int flags,struct sockaddr *from,int *fromlen);

unsigned __LONG32 inet_addr(const char *cp);

char* inet_ntoa(struct in_addr in);

u_long htonl(u_long hostlong);

u_short htons(u_short hostshort);

u_long ntohl(u_long netlong);

u_short ntohs(u_short netshort);

struct sockaddr {
    u_short sa_family;
    char    sa_data[14];
};

struct sockaddr_in {
    short   sin_family;
    u_short sin_port;
    struct in_addr  sin_addr;
    char    sin_zero[8];
};
addr.sin_addr.s_addr

typedef struct in_addr {
  union {
    struct { u_char  s_b1, s_b2, s_b3, s_b4; } S_un_b;
    struct { u_short s_w1, s_w2; } S_un_w;
    u_long S_addr;
  } S_un;
} IN_ADDR, *PIN_ADDR, *LPIN_ADDR;

#define s_addr  S_un.S_addr
```

5、Windwos系统下的TCP客户端

```
#include <stdio.h>
#include <winsock2.h>

int main()
{
    // 用户存储winsock库的信息
    WSADATA wsaData;
    WSAStartup(0x0202,&wsaData);

    // 创建socket对象
    SOCKET sock = socket(AF_INET,SOCK_STREAM,0);
    if(0 > sock)
    {
        perror("socket");
        return -1;
```

```
    }

    // 准备通信地址
    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(12345);
    addr.sin_addr.s_addr = inet_addr("192.168.56.103");
    int addrlen = sizeof(addr);

    // 连接服务器
    if(connect(sock,(struct sockaddr*)&addr,addrlen))
    {
        perror("connect");
        return -1;
    }

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        printf(">>>");
        scanf("%s",buf);

        int ret = send(sock,buf,strlen(buf)+1,0);
        if(0 >= ret || 0 == strcmp("quit",buf))
            break;

        ret = recv(sock,buf,buf_size,0);
        if(0 >= ret)
            break;

        printf("recv:%s byte:%d\n",buf,ret);
    }

    printf("通信结束！\n");
    // 关闭socket对象
    closesocket(sock);
    // 注销winsock库
    WSACleanup();
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
#include <tcp_socket.h>

TcpSocket* ts;

void sigint(int signum)
{
    close_tcp(ts);
    exit(1);
}

```

```c
int main(int argc,const char* argv[])
{
    signal(SIGINT,sigint);

    ts = create_tcp(argv[1],atoi(argv[2]));
    if(NULL == ts)
        return -1;

    if(listen_tcp(ts))
        return -1;

    TcpSocket* tsc;
    for(;;)
    {
        tsc = accept_tcp(ts);
        if(NULL == tsc)
            continue;

        // 创建一个子进程并跳出当前循环
        if(0 == fork())
            break;
    }

    char buf[4096];
    size_t buf_size = sizeof(buf);

    // 子进程为客户端服务
    for(;;)
    {
        int ret = recv_str_tcp(tsc,buf);
        if(0 >= ret || 0 == strcmp("quit",buf))
            break;

        printf("recv:%s byte:%d\n",buf,ret);
        strcat(buf,":return");

        ret = send_str_tcp(tsc,buf);
        if(0 >= ret)
            break;
    }

    printf("客户端退出！\n");
    close_tcp(tsc);

    return 0;
}

// gcc tcp_server.c -lsocket -o tcp_server
// ./tcp_server 192.168.56.103 12345
```

6、Windwos系统下的UDP客户端

```c
#include <stdio.h>
#include <winsock2.h>

int main(int argc,const char* argv[])
{
```

```
    // 用户存储winsock库的信息
    WSADATA wsaData;
    WSAStartup(0x0202,&wsaData);

    // 创建socket对象
    SOCKET sock = socket(AF_INET,SOCK_DGRAM,0);
    if(0 > sock)
    {
        perror("socket");
        return -1;
    }

    // 准备通信地址(目标的)
    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(12345);
    addr.sin_addr.s_addr = inet_addr("192.168.56.103");
    int addrlen = sizeof(addr);

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        printf(">>>");
        scanf("%s",buf);
        int ret = sendto(sock,buf,strlen(buf)+1,0,(struct sockaddr*)&addr,addrlen);
        if(0 > ret || 0 == strcmp("quit",buf))
        {
            break;
        }

        ret = recvfrom(sock,buf,buf_size,0,(struct sockaddr*)&addr,&addrlen);
        printf("from:%s recv:%s byte:%d\n",inet_ntoa(addr.sin_addr),buf,ret);
        if(0 > ret)
        {
            break;
        }
    }

    printf("通信结束！\n");
    // 关闭socket对象
    closesocket(sock);
    // 注销winsock库
    WSACleanup();
}
```

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

```

```c
int main(int argc,const char* argv[])
{
    // 创建socket对象
    int sockfd = socket(AF_INET,SOCK_DGRAM,0);
    if(0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    // 准备通信地址(本机的)
    struct sockaddr_in addr = {}, src_addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(12345);
    addr.sin_addr.s_addr = inet_addr("192.168.56.103");
    socklen_t addrlen = sizeof(addr);

    // 绑定地址与socket对象
    if(bind(sockfd,(struct sockaddr*)&addr,addrlen))
    {
        perror("bind");
        return -1;
    }

    char buf[4096];
    size_t buf_size = sizeof(buf);

    for(;;)
    {
        // 接收数据
        int size = recvfrom(sockfd,buf,buf_size,0,(struct sockaddr*)&src_addr,&addrlen);

        printf("from:%s recv:%s byte:%d\n",inet_ntoa(src_addr.sin_addr),buf,size);
        strcat(buf,":return");

        // 返回数据
        sendto(sockfd,buf,strlen(buf)+1,0,(struct sockaddr*)&src_addr,addrlen);
    }

    return 0;
}
```

7、Windows系统的Socket与Linux系统的Socket的区别

Linux系统下把一切都抽象成了文件，这样的好处是给所有的硬件都提供了一套统一的操作接口，所以Linux系统下的Socket操作其核心本质就是文件操作，创建Socket对象时，就是创建一个隐藏的Socket文件，Linux系统返回的就是文件描述符，Socket通信时就是文件读写，所以在Linux系统的Socket可以使用read、write、close、fwrite、fread、fscanf、fprintf、fgets、fclose函数。

Windows系统中的Socket是内核对象，创建Socket就是内核中创建一个内核对象，返回的是使用Socket的凭证，在Windows系统下被称为句柄，所以Windows系统的Socket对象必须使用closesocket关闭，而不是像Linux系统的Socket一样使用close。

十、多路复用

1、IO模型介绍

1、阻塞IO，常用的scanf、printf、cout、cin、recv、recvfrom

2、非阻塞IO，读取不到数据也立即返回

recv、send和flags设置为MSG_DONTWAIT

open打开文件时带O_NONBLOCK标志，read、write就是非阻塞的

3、信号驱动IO(Qt的信号和槽机制)

4、异步IO(不能立即得到结果，但可以先去做其他事情，能完成后再通知你)

5、多路复用IO

```
在不创建新的进程和线程的情况下监控多个文件描述符，多应用于网络编程时一个服务端程序为多个客户端程序提供服务，多用于在高并发但业务逻辑简单，客户端需要的服务时间短，响应时间无太高要求的场景。
```

2、使用select函数实现多路复用IO

```
int select(int nfds, fd_set *readfds, fd_set *writefds,fd_set *exceptfds, struct timeval *timeout);

功能：监控多个文件描述符的 读、写、异常等操作
nfds：最大文件描述符+1
readfds：监控读操作文件描述符集合，并且会把监控的结果存储到readfds，既输入又输出的参数。
writefds：监控写操作文件描述符集合，同上
exceptfds：监控异常操作文件描述符集合，同上
timeout：设置超时时间
返回值：监控到文件描述符的个数,超时返回0，出错返回-1

void FD_CLR(int fd, fd_set *set);
功能：从集合吕删除文件描述符

int  FD_ISSET(int fd, fd_set *set);
功能：测试集合中是否有文件描述符存在

void FD_SET(int fd, fd_set *set);
功能：向集合中添加文件描述符

void FD_ZERO(fd_set *set);
功能：清空文件描述符集合
```

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main()
```

```
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5678);
    addr.sin_addr.s_addr = inet_addr("192.168.0.111");
    socklen_t addrlen = sizeof(addr);

    if (bind(sockfd, (struct sockaddr *)&addr, addrlen))
    {
        perror("bind");
        return -1;
    }

    if (listen(sockfd, 10))
    {
        perror("listen");
        return -1;
    }

    // 创建两个文件描述符集合
    fd_set readfd, readcp;
    // 清空两个文件描述符集合
    FD_ZERO(&readfd);
    FD_ZERO(&readcp);
    // 把sockfd加到readfd中
    FD_SET(sockfd, &readfd);

    int maxfd = sockfd + 1;
    struct timeval timeout = {5, 5000000};
    char buf[4096] = {};

    for (;;)
    {
        // 每次都要重新向select传递所监控的对象
        readcp = readfd;

        // 监控集合中的文件描述符
        int ret = select(maxfd, &readcp, NULL, NULL, &timeout);
        // 返回值小于零出错，等于零超时
        if (ret <= 0)
            continue;

        for (int i = 3; i < maxfd; i++)
        {
            // 检查sockfd 是否有新客户端连接
            if (i == sockfd && FD_ISSET(sockfd, &readcp))
            {
                int clifd = accept(sockfd, (struct sockaddr *)&addr, &addrlen);
                if (clifd < 0)
                    continue;
```

```

                FD_SET(clifd, &readfd);
                if (clifd + 1 >= maxfd)
                {
                    maxfd = clifd + 1;
                }
            }
            else if (FD_ISSET(i, &readcp))
            {
                int ret = recv(i, buf, sizeof(buf), 0);
                if (ret <= 0)
                {
                    FD_CLR(i, &readfd);
                    continue;
                }
                // 这一块的任务必须能快速处理，否则其他所有客户端都要等着
                printf("fd:%d recv:%s\n", i, buf);
                send(i, buf, strlen(buf) + 1, 1);
            }
        }
    }
}
```

select设计不合理的地方：

1、所有被监视的文件描述符都需要检查（效率不高）。

2、每次调用select都需要向它传递新的监视对象信息。

select的优点是：

程序的兼容性高，所有操作系统都支持。

4、使用pselect函数实现多路复用IO

```
int pselect(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset, const struct timespec *timeout, const sigset_t *sigmask);

功能：与select的功能大致类似
区别：
    1、select函数用的timeout参数，是一个timeval的结构体（包含秒和微秒），然而pselect用的是一个timespec结构体（包含秒和纳秒）
    2、select函数可能会为了指示还剩多长时间而更新timeout参数，然而pselect不会改变timeout参数
    3、select函数没有sigmask参数，当pselect的sigmask参数为null时，两者行为时一致的。有sigmask的时候，pselect相当于如下的select()函数，在进入select()函数之前手动将信号的掩码改变，并保存之前的掩码值；select()函数执行之后，再恢复为之前的信号掩码值。
```

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>

```

```c
int main()
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5678);
    addr.sin_addr.s_addr = inet_addr("192.168.0.111");
    socklen_t addrlen = sizeof(addr);

    if (bind(sockfd, (struct sockaddr *)&addr, addrlen))
    {
        perror("bind");
        return -1;
    }

    if (listen(sockfd, 10))
    {
        perror("listen");
        return -1;
    }

    // 创建两个文件描述符集合
    fd_set readfd, readcp;
    // 清空两个文件描述符集合
    FD_ZERO(&readfd);
    FD_ZERO(&readcp);
    // 把sockfd加到readfd中
    FD_SET(sockfd, &readfd);

    int maxfd = sockfd + 1;
    // struct timeval timeout = {5,5000000};
    struct timespec timeout = {5, 500000000};
    char buf[4096] = {};

    for (;;)
    {
        // 每次都要重新向select传递所监控的对象
        readcp = readfd;

        // 监控集合中的文件描述符
        // pselect 不会屏蔽信号，需要把已经屏蔽的信号集经pselect函数，pselect会把屏蔽的信号取消，等执行结束后再还原屏蔽的信号
        int ret = pselect(maxfd, &readcp, NULL, NULL, &timeout, NULL);
        // 返回值小于零出错，等于零超时
        if (ret <= 0)
            continue;

        for (int i = 3; i < maxfd; i++)
        {
            // 检查sockfd 是否有新客户端连接
            if (i == sockfd && FD_ISSET(i, &readcp))
            {
```

```
                int clifd = accept(sockfd, (struct sockaddr *)&addr, &addrlen);
                if (clifd < 0)
                    continue;

                FD_SET(clifd, &readfd);
                if (clifd + 1 >= maxfd)
                {
                    maxfd = clifd + 1;
                }
            }
            else if (FD_ISSET(i, &readcp))
            {
                int ret = recv(i, buf, sizeof(buf), 0);
                if (ret <= 0)
                {
                    FD_CLR(i, &readfd);
                    continue;
                }
                printf("fd:%d recv:%s\n", i, buf);
                send(i, buf, strlen(buf) + 1, 1);
            }
        }
    }
}
```

5、使用poll函数实现多路复用IO

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout);
fds：所有被监控的文件描述符结构体数组
nfds：数组的长度
timeout：超时时间，毫秒单位

struct pollfd
{
    int fd;   //被监控文件描述符
    short events; // 等待的需要监控事件
    short revents; // 实际发生了的事件，也就是返回结果
};

events:
    POLLIN     普通或优先级带数据可读
    POLLPRI    高优先级数据可读
    POLLOUT    普通数据可写
    POLLRDHUP   对方socket关闭
    POLLERR    发生错误
    POLLHUP    发生挂起
    POLLNVAL    描述字不是一个打开的文件

poll特点：
    1.文件描述符没有最大限制 -数据结构：链表
    2.每次调用都需要将fd集合从用户态拷贝到内核态
    3.内核需要遍历所有fd，效率低
```

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <poll.h>

int main()
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5678);
    addr.sin_addr.s_addr = inet_addr("192.168.0.111");
    socklen_t addrlen = sizeof(addr);

    if (bind(sockfd, (struct sockaddr *)&addr, addrlen))
    {
        perror("bind");
        return -1;
    }

    if (listen(sockfd, 10))
    {
        perror("listen");
        return -1;
    }

    // 数组的长度,已经使用元素的最大小
    nfds_t nfds = 10 ,maxi = 1;

    struct pollfd *fds = calloc(sizeof(struct pollfd), nfds);
    fds[0].fd = sockfd;
    fds[0].events = POLLIN;

    for (;;)
    {
        int ret = poll(fds, nfds, 100000);
        if(0 >= ret)
            continue;

        if (fds[0].revents & POLLIN)
        {
            int clifd = accept(sockfd, (struct sockaddr *)&addr, &addrlen);
            if (clifd < 0)
                continue;

            // 寻找一个位置
            for (int i = 0; i < nfds; i++)
            {
```

```
                if (0 == fds[i].fd)
                {
                    fds[i].fd = clifd;
                    fds[i].events = POLLIN;
                    if(i >= maxi)
                        maxi = i+1;
                    break;
                }
            }
        }
        for (int i = 1; i < maxi; i++)
        {
            if (fds[i].revents & (POLLIN))
            {
                char buf[4096] = {};
                int ret = recv(fds[i].fd, buf, sizeof(buf), 0);
                if (ret <= 0)
                {
                    fds[i].fd = 0;
                    continue;
                }
                printf("clifd:%d recv:%s\n", fds[i].fd, buf);
                send(fds[i].fd, buf, strlen(buf) + 1, 1);
            }
        }
    }
}
```

6、使用epoll函数实现多路复IO

```
int epoll_create(int size);
功能：创建用于保存被监控文件描述符的空间
返回值：epoll内核对象的描述符

int epoll_ctl(int epfd, int  op,  int  fd,  struct epoll_event *event);
epfd：epoll内核对象的描述符
op：
    EPOLL_CTL_ADD 向epoll对象中添加要监控的描述符
    EPOLL_CTL_MOD 修改已添加的描述符的监控事件
    EPOLL_CTL_DEL 删除epoll对象中描述符

功能：向文件中添加、删除，文件描述符

int epoll_wait(int epfd, struct epoll_event *events,int maxevents, int timeout);
功能：监控文件描述符
返回值：监控事件发生了的描述符数量

特点：
    1.文件描述符没有最大限制 --数据结构：红黑树
    2.只需拷贝一次fd到内核态
    3.内核只需判断就绪链表是否为空，不需要遍历所有fd，效率高，并把就绪fd拷贝到用户空间
```

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/epoll.h>

int main()
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (0 > sockfd)
    {
        perror("socket");
        return -1;
    }

    struct sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(6666);
    addr.sin_addr.s_addr = inet_addr("192.168.0.111");
    socklen_t addrlen = sizeof(addr);

    if (bind(sockfd, (struct sockaddr *)&addr, addrlen))
    {
        perror("bind");
        return -1;
    }

    if (listen(sockfd, 10))
    {
        perror("listen");
        return -1;
    }

    int epfd = epoll_create(10);

    struct epoll_event event;
    event.data.fd = sockfd;
    event.events = EPOLLIN;

    epoll_ctl(epfd, EPOLL_CTL_ADD, sockfd, &event);

    // 存储监控结果的数组
    struct epoll_event *events = calloc(sizeof(struct epoll_event), 10);

    for (;;)
    {
        int cnt = epoll_wait(epfd, events, 10, 100000);
        if (cnt <= 0)
            continue;

        for (int i = 0; i < cnt; i++)
        {
            printf("cnt=%d fd=%d sockfd:%d\n", cnt, events[0].data.fd, sockfd);
```

```
            if (events[i].data.fd == sockfd)
            {
                int clifd = accept(sockfd, (struct sockaddr *)&addr, &addrlen);
                if (0 > clifd)
                    continue;
                event.data.fd = clifd;
                event.events = EPOLLIN;
                epoll_ctl(epfd, EPOLL_CTL_ADD, clifd, &event);
            }
            else
            {
                char buf[4096] = {};
                int ret = recv(events[i].data.fd, buf, sizeof(buf), 0);
                if (ret <= 0)
                {
                    epoll_ctl(epfd, EPOLL_CTL_DEL, events[i].data.fd, events + i);
                    continue;
                }

                printf("clifd:%d recv:%s\n", events[i].data.fd, buf);
                send(events[i].data.fd, buf, strlen(buf) + 1, 1);
            }
        }
    }
}
```